

A PROJECT REPORT ON

**Multi-Party Crypto Vault System for Secure
Digital Transaction**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Rushi Balapure	Exam No: B400050332
Prachi Patil	Exam No: B400050538
Sameer Patil	Exam No: B400050543
Samiksha Sarode	Exam No: B400050577



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled
Multi-Party Crypto Vault System for Secure Digital Transaction

Submitted by

Rushi Balapure
Prachi Patil
Sameer Patil
Samiksha Sarode

Exam No: B400050332
Exam No: B400050538
Exam No: B400050543
Exam No: B400050577

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Dr. S. N. Girme** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Dr. S. N. Girme
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune

Date: / /

ACKNOWLEDGEMENT

It gives us great pleasure in presenting the project report on ‘Multi-Party Crypto Vault System for Secure Digital Transactions’.

*We would like to thank our project guide **Dr. Shital Girme** for giving us all the help and guidance we needed. We are really grateful to her for her kind support and valuable suggestions.*

*We are also grateful to **Dr G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE Project Coordinator, for her timely guidance and support.*

**RUSHI BALAPURE
PRACHI PATIL
SAMEER PATIL
SAMIKSHA SARODE
(B.E. COMPUTER ENGINEERING)**

ABSTRACT

Secure key management remains a fundamental challenge in cryptographic systems, especially within decentralized environments where trust must be distributed and single points of failure eliminated. Traditional approaches often rely on centralized control or individual custodianship, which can be vulnerable to compromise, loss, or misuse. To address these challenges, this project implements a multi-party crypto vault that uses a threshold-based secret sharing mechanism to enhance the security, reliability, and resilience of cryptographic key management.

In our approach, a sensitive cryptographic key is divided into multiple fragments, or shards, using Shamir's Secret Sharing Scheme. These shards are distributed among multiple independent parties, such that the original key can only be reconstructed when a predefined quorum of parties collaborates. This ensures that no single party holds enough information to access or misuse the key independently. As a result, the system offers strong protection against insider threats and external compromise.

The report discusses the architecture and design of the vault system, covering shard generation, distribution, and secure reconstruction. It highlights the use of AWS Lambda functions to perform computation in a stateless, serverless environment, ensuring that no shard is stored permanently on any storage medium. Email-based delivery using AWS SES is used to securely transmit key shares to designated parties.

We analyze the mathematical foundation of the scheme and evaluate its resilience against various attack vectors, including collusion, rogue participant behavior, and partial key exposure. Furthermore, we assess system performance through metrics such as computational overhead, transaction latency, and the time required for secure reconstruction.

Our experimental results demonstrate that this approach achieves a favorable balance between security, efficiency, and scalability, making it suitable for secure transaction authorization in blockchain networks, decentralized applications, and other cryptographic systems. The implementation confirms that multi-party key management is a practical and secure alternative to conventional models in trust-minimized environments.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition & Objectives	3
1.3.1	Problem Definition	3
1.3.2	Objectives	4
1.4	Project Scope & Limitations	4
1.4.1	Scope	4
1.4.2	Limitations	5
1.5	Methodologies of Problem solving	6
1.5.1	Multi-Party Techniques (Multi-Sig, MPC)	6
1.5.2	Key Management Models	8
1.5.3	Fault Tolerance and Recovery Mechanisms	9
2	Literature Survey	10
3	Software Requirements Specification	15
3.1	Assumptions and Dependencies	16
3.1.1	Assumptions	16
3.1.2	Dependencies	16
3.2	Functional Requirements	17
3.2.1	User Interaction Module	17
3.2.2	Key Management Module	17
3.2.3	Transaction Processing Module	17
3.2.4	Blockchain Integration Module	17
3.2.5	Fault Tolerance Recovery Module	18
3.2.6	Vault Administration Module	18
3.3	Nonfunctional Requirements	18
3.3.1	Performance Requirements	18
3.3.2	Safety Requirements	18
3.3.3	Security Requirements	19

3.3.4	Software Quality Attributes	19
3.4	External Interface Requirements	19
3.4.1	User Interfaces	19
3.4.2	Software Interfaces	20
3.5	Analysis Models: SDLC Model to be applied	20
4	System Design	21
4.1	System Architecture	22
4.2	Mathematical Model	23
4.2.1	Shard Generation Using Shamir's Secret Sharing	23
4.2.2	Shard Reconstruction via Lagrange Interpolation	24
4.2.3	Enhanced Multi-Secret Sharing Scheme	24
4.2.4	Advantages of the Enhanced Implementation	25
4.3	Data Flow Diagram	26
4.4	UML Diagram	29
4.4.1	Use Case Diagram	29
4.4.2	Activity Diagram	31
4.4.3	Sequence Diagram	32
5	Project Plan	34
5.1	Project Estimate	35
5.1.1	Reconciled Estimates	35
5.1.2	Project Resources	36
5.2	Risk Management	37
5.2.1	Risk Identification	37
5.2.2	Risk Analysis	37
5.2.3	Overview of Risk Mitigation, Monitoring, Management	38
5.3	Project Schedule	38
5.3.1	Project Task Set	38
5.3.2	Timeline Chart	39
5.4	Team Organization	39
5.4.1	Team structure	39
6	Project Implementation	40
6.1	Overview of Project Modules	41
6.2	Tools and Technologies Used	41
6.3	Algorithm Details	42
6.3.1	Transaction Processing Algorithm	42
6.3.2	User Interaction Algorithm	43
6.3.3	Key Management Algorithm	43
6.3.4	Blockchain Network Algorithm	44

6.3.5	Vault Administration Algorithm	44
7	Software Testing	45
7.1	Types of Testing	46
7.1.1	Based on Testing Approach	46
7.1.2	Based on Development Phase	46
7.1.3	Functional & Non-Functional Testing	46
7.2	Test Cases and Test Results	47
8	Results	52
8.1	Outcomes	53
8.2	Results and Outputs	53
8.2.1	Frontend	53
8.3	Results Analysis	62
8.3.1	Comparative Analysis	62
9	Conclusion	63
9.1	Applications	64
9.2	Advantages	65
9.3	Future Scope and Recommendations	65
9.4	Technical Observations	66
9.5	Learnings and Reflections	66
9.6	Final Remarks	67
10	Appendix	68
10.1	Appendix A	69
10.1.1	Problem Statement Feasibility Assessment	69
10.2	Appendix B	71
10.3	Appendix C	75
10.3.1	Plagiarism Report	75
11	References	76

List of Figures

4.1	System Architecture	22
4.2	Data Flow Diagram - Level 0	26
4.3	Data Flow Diagram - Level 1	27
4.4	Use Case Diagram	29
4.5	Activity Diagram	31
4.6	Sequence Diagram	32
5.1	Timeline Chart	39
8.1	Home Page	53
8.2	Team Page	54
8.3	Team1: Vault Creation	54
8.4	Team1: Email Input	55
8.5	Team1: Details - 1	55
8.6	Team1: Details - 2	56
8.7	Team2: Vault Creation	56
8.8	Team2: Email Input	57
8.9	Team2: Details - 1	57
8.10	Team2: Details - 2	58
8.11	User Profile - 1	58
8.12	User Profile - 2	59
8.13	User Profile - 3	59
8.14	Buy Assets	60
8.15	Sell Assets	60
8.16	Receive Assets - 1	61
8.17	Receive Assets - 2	61
8.18	Send Assets	62
10.1	Survey Paper	72
10.2	Implementation Paper	74
10.3	Plagiarism Report	75

CHAPTER 1

INTRODUCTION

1.1 Overview

The emergence of blockchain technology and cryptocurrencies has revolutionized digital asset management, enabling decentralized ecosystems that eliminate the need for traditional intermediaries like banks. Cryptocurrencies and DeFi platforms have empowered individuals and organizations with full asset control, enhancing privacy and independence. However, this autonomy introduces risks such as theft, loss, or key compromise, especially as asset value grows. To address these challenges, secure, decentralized vault systems are essential to prevent unauthorized access and ensure fault tolerance. Traditional centralized vaults pose a single point of failure, making them vulnerable to threats.

This project implements a secure, decentralized multi-party crypto vault system utilizing blockchain technology and cryptographic techniques. By employing Multi-Party Computation (MPC), the system ensures that private keys are never fully reconstructed, distributing control among multiple participants. Smart contracts govern transaction authorization, and a user-friendly interface facilitates interaction with the vault. The design emphasizes fault tolerance, privacy, and resilience against single points of failure, offering a robust solution for secure digital asset management.

1.2 Motivation

The rapid adoption of blockchain-based technologies and the growing prominence of decentralized finance (DeFi) platforms have highlighted the need for enhanced security mechanisms. With the increasing value of digital assets, such as cryptocurrencies, and the expansion of decentralized ecosystems, there is a growing risk of attacks targeting individual users and custodial services. Traditional approaches to securing digital assets often involve relying on a single custodian or key holder, which can lead to disastrous consequences if the custodian's security is compromised. This single point of failure makes systems vulnerable to theft, loss, or key mismanagement, resulting in irrecoverable financial losses.

The motivation behind implementing a multi-party crypto vault system is to address these vulnerabilities by decentralizing control and removing reliance on a single trusted entity. By distributing authority among multiple participants, the system enhances security, preserves privacy, and introduces fault tolerance. Our implementation adopts Multi-Party Computation (MPC), which enables secure, distributed keyless authorization without exposing private keys. MPC allows multiple parties to collaboratively approve

transactions through encrypted computation, ensuring confidentiality and integrity. Unlike traditional approaches, our design prevents any individual or compromised party from misusing access. This makes it especially effective in high-value digital asset management where accountability, transparency, and resilience are critical. The use of smart contracts further automates and enforces secure coordination, making the vault system suitable for real-world deployment in decentralized financial environments.

1.3 Problem Definition & Objectives

1.3.1 Problem Definition

The primary challenge in designing a multi-party crypto vault system is striking the right balance between security, privacy, scalability, and efficiency. In traditional systems, the reliance on a single custodian or key holder presents a single point of failure, which is a significant security risk. Conversely, distributing control among multiple participants can enhance security, but it also introduces new challenges. One of the main concerns is how to ensure efficient transaction approval while maintaining privacy and scalability. Multi-Sig schemes provide a solution for distributed control by requiring multiple parties to sign off on transactions. However, they come with privacy limitations, as the participants' keys are exposed during the signing process. This makes Multi-Sig suitable for applications where privacy is not the primary concern, but where efficiency and ease of implementation are critical.

On the other hand, MPC offers a more privacy-preserving solution, enabling participants to jointly authorize transactions on encrypted data without revealing their private keys. While this enhances security and privacy, MPC introduces significant computational and communication overhead. The additional resources required for MPC can result in delays, making it less suitable for real-time or high-frequency applications. Another challenge lies in managing the keys and ensuring fault tolerance, so that the system remains operational even in cases where participants lose their keys or fail to cooperate. This report evaluates the challenges and trade-offs involved in Multi-Sig and MPC approaches for building secure, scalable, and privacy-preserving multi-party crypto vaults, and presents the implementation of a vault system based on the MPC approach due to its stronger privacy guarantees and decentralized authorization model.

1.3.2 Objectives

- To compare the performance, scalability, and security properties of the cryptographic methods—Multi-Sig and MPC in particular—that are employed in the creation of multi-party crypto vaults.
- To evaluate different key management strategies, such as split-key management and blockchain-based verification, which ensure that no single participant holds complete control over the assets.
- To review fault tolerance techniques, such as time-locked withdrawal procedures that postpone transaction execution to avoid unwanted access and social recovery models, which allow trusted contacts to help with key recovery.
- To identify the strengths and weaknesses of both Multi-Sig and MPC approaches in various use cases, such as high-frequency blockchain wallets, custodial services, and institutional asset management.
- To propose an MPC-based system architecture designed for high-security, privacy-sensitive applications, focusing on reducing latency and improving the scalability of MPC protocols.
- To explore future research areas, including the integration of post-quantum cryptography to prepare for emerging threats and the optimization of MPC protocols to improve their efficiency in real-world applications.

1.4 Project Scope & Limitations

1.4.1 Scope

This project focuses on the design and implementation of a secure, threshold-based multi-party crypto vault system using a secret sharing scheme to manage cryptographic keys in a decentralized manner. The implementation is built around Shamir's Secret Sharing (SSS), with secure key distribution and reconstruction facilitated through AWS Lambda functions and AWS SES for stateless, ephemeral operations.

The primary scope includes:

- Implementing a key generation and sharding mechanism using threshold cryptography to split a secret key among n participants, requiring a quorum of k members for reconstruction.

- Developing secure Lambda functions for:
 - Distributing key shards to users via email.
 - Reconstructing the original key upon quorum agreement.
- Ensuring no persistent storage of key shards, aligning with MPC-inspired security principles that emphasize ephemeral data flow and minimal trust assumptions.
- Validating system robustness against common threats such as:
 - Collusion of fewer than k participants.
 - Non-cooperation or loss of key shares by some parties.
 - Unauthorized key reconstruction attempts.
- Evaluating the system for security, fault tolerance, computational overhead, and reconstruction latency through experimentation.

This scope aligns with the project's objective of contributing to decentralized key management research by offering a lightweight, practical, and secure architecture that minimizes reliance on centralized trust while ensuring high levels of security and privacy.

1.4.2 Limitations

Despite its practical benefits, the project has certain limitations, which are acknowledged below:

- **Complex Implementation:** The deployment of multi-party vaults involves complex cryptographic protocols of MPC (Multi-Party Computation), which can be challenging to implement correctly. This complexity might require specialized knowledge and resources.
- **Coordination Challenges:** Multi-party vaults require coordination among participants to authorize transactions. Delays can occur if one or more participants are unavailable or slow to respond, which could hinder timely execution of transactions.
- **Increased Latency:** Due to the requirement of gathering key shards and securing consensus from multiple parties, transactions may experience increased latency compared to traditional, single-party wallets. This might not be suitable for scenarios demanding high transaction throughput.

- **Vulnerability to Collusion:** While the system protects against unilateral control, it remains vulnerable to collusion among participants. If enough parties collaborate maliciously, they can compromise the vault and gain unauthorized access to the digital assets.

1.5 Methodologies of Problem solving

This section presents the key cryptographic techniques, key management strategies, and fault tolerance mechanisms essential for building a multi-party crypto vault system. We explore the Multi-Signature (Multi-Sig) and Multi-Party Computation (MPC) techniques, followed by key management models, and conclude with fault tolerance and recovery mechanisms.

1.5.1 Multi-Party Techniques (Multi-Sig, MPC)

Both Multi-Signature (Multi-Sig) and Multi-Party Computation (MPC) are used to distribute control and ensure security in multi-party crypto vault systems. This section discusses their mechanisms, similarities, differences, and the most relevant methods for crypto vaults.

1. Multi-Signature (Multi-Sig) Approach:

- **How It Works:**
 - A multi-signature (multi-sig) scheme ensures that multiple parties must sign a transaction or operation for it to be executed.
 - In a typical threshold multi-sig model, an m-out-of-n scheme is employed, meaning at least m out of n participants must provide their signatures for the operation to proceed.
 - Threshold ECDSA and Schnorr signatures aggregate partial signatures from different participants into a single valid signature, ensuring that no single party can act unilaterally.
 - Bloom filter-based wallets further optimize multi-sig transactions by reducing storage overhead. The Bloom filter stores participant information compactly, ensuring that the network can quickly verify multi-sig transactions without downloading extensive data.
- **Advantages:**

- Fast and scalable: Well-suited for high-frequency transactions.
- Simple to implement: Uses relatively simple cryptographic protocols.
- **Challenges:**
 - Lower privacy: Key inputs are visible during signing.
 - Collusion risk: Requires trust among participants.

2. Multi-Party Computation (MPC) Approach:

- **How It Works:**
 - MPC enables multiple participants to jointly compute a function on private inputs without revealing those inputs to one another. This ensures data privacy during collaborative operations.
 - In the context of a crypto vault, participants collaborate to authorize transactions by encrypting their inputs (such as private keys or votes) and processing them through an MPC protocol.
 - One common method is threshold MPC with lightweight FHE, where only the required subset of participants collaborates to compute the output. Fully Homomorphic Encryption (FHE) allows operations to be performed directly on encrypted data, meaning sensitive information is never exposed during computation.
 - For low-latency applications, one-round threshold ECDSA with MPC is used. This approach allows participants to generate the signature in a single communication round, reducing interaction overhead.
- **Advantages:**
 - High privacy and security: Inputs remain encrypted throughout the process.
 - Distributed trust: Reduces reliance on a single trusted party.
- **Challenges:**
 - High computational overhead: Encryption adds latency, limiting real-time performance.
 - Complex to implement: Requires advanced cryptographic techniques.

1.5.2 Key Management Models

Effective key management is essential for securing access to the crypto vault. This section describes the two primary models used in crypto vaults: Split-Key Management and Blockchain-Based Key Management.

1. Split-Key Management and Group Keys:

- **Mechanism:**
 - The cryptographic key is split into fragments and distributed among participants using threshold cryptography. A predefined threshold number of fragments must be combined to reconstruct the key. If some participants lose access to their fragments, the vault can still be accessed by the remaining members meeting the threshold.
- **Advantages:**
 - Decentralized control: No single party holds the complete key.
 - Fault tolerance: Ensures access even if some keys are lost.

2. Blockchain-Based Key Management:

- **Mechanism:**
 - Public key hashes are stored on an immutable blockchain ledger. A permissioned blockchain verifies key ownership and automates access control through smart contracts. These contracts enforce multi-signature rules and enable secure governance of vault operations.
- **Advantages:**
 - Tamper-proof storage: Blockchain ensures the authenticity of keys.
 - Automation: Smart contracts streamline processes, such as multi-signature approvals.

3. Hybrid Key Management Model:

- **Mechanism:**
 - Combines split-key management with blockchain-based key verification. Keys are split among participants, and public key hashes are stored on the blockchain. Smart contracts govern multi-signature transactions and automate recovery mechanisms.

- **Advantages:**
 - Enhanced security and decentralization: Prevents unauthorized access while ensuring accessibility.
 - Scalability: Suitable for large multi-party setups.

1.5.3 Fault Tolerance and Recovery Mechanisms

Fault tolerance mechanisms ensure that the vault remains secure and operational even in cases of key loss or participant failure. Two key recovery mechanisms are proposed:

1. Social Network-Based Key Recovery:

- **Mechanism:**
 - Shamir's Secret Sharing is used to distribute key shares among trusted contacts in the user's social network. If the key is lost, a threshold subset of these contacts collaborates to restore it.
- **Advantages:**
 - Useful for personal wallets where key loss is a common risk.
 - Decentralizes trust by involving multiple participants in the recovery process.

2. Vault Custody Protocols with Time-Locked Withdrawals:

- **Mechanism:**
 - A time-lock is placed on transactions, introducing a delay (e.g., 24-48 hours) before the assets are transferred. During this time, participants can cancel the transaction if it is unauthorized or suspicious.
- **Advantages:**
 - Ideal for high-security vaults where unauthorized access could lead to significant loss.
 - Provides an opportunity for collective decision-making during the delay period.

CHAPTER 2

LITERATURE SURVEY

Table 2.1: Literature Survey Table

Author	Key Findings	Limitations
Y Ji, Y Xiao, et.al -2023	- Improves data security by using adaptor signatures. - With traditional signatures, key loss is a major challenge. - A multi-party agreement increases confidence in the system making data more secure.	- Increases complexity and overhead. - High cost for big systems. - Uncertainty in post-quantum usability.
P Zhang, F Ge, et.al. -2023	- MEMS was suggested to protect from k-sum assaults. - Public Third Party (PTP) ensures secure commitment. - Improved performance using centralized communication.	- PTP security is necessary. - Efficiency in larger networks may be affected because of inadequate testing. - Mostly focused on k-sum assaults.
V Di Nicola, R Longo, et.al. -2023	- Threshold multisignature which is independent of blockchain with 2 out of 3 essential requirements. - Improved protection on all blockchains.	- Simplifies complex MPC methods. - For improved performance, offline alternatives are suggested. - Increased costs of communication and computing for scalability purposes.

Author	Key Findings	Limitations
J Han, M Song, , et.al. -2021	• For better validation, T-ECDSA combines several signatures into one. • Bloom filters protect participant's privacy while reducing transaction sizes. • Effective without modifying the blockchain protocol.	• Collisions using bloom filters may result in false positives. • Problems with storage and accuracy for larger applications. • For larger setups, a static threshold reduces flexibility.
Z Guan, X Zhou, et.al. - 2021	• Ring signatures give dual-sided privacy protection. • Combining blockchain increases trust and security.	• Ring signatures result in high computational overhead. • There is a fixed public key limit and communication overhead.
J Zhou, Y Feng, Z Wang, D Guo -2021	• MPC is used in a permissioned blockchain to boost privacy. • Zero-knowledge proofs, secret sharing, and homomorphic encryption are used. • Uses Hyperledger Fabric integration.	• Scalability is affected by communication. • Dependency on trusted processors increases trust issues. • Complicated tasks are reduced by limited homomorphic encryption.
R Genaro, S Goldfeder -2020	• One-round cutoff ECDSA boosts the usability of cryptocurrencies. • Reduced rounds of signature interactions.	• Key generation has to be restarted when preprocessing is abandoned. • Zero-knowledge proofs increase complexity.

Author	Key Findings	Limitations
J Swambo, S Hommel, B McElrath, B Bishop -2020	• Fund transfers with multi-signature wallets need multiple approvals. • Time-locked, pre-signed transactions help in recovery in case of crucial loss circumstances. • In case of theft transactions are delayed to ensure safe recovery.	• For users unfamiliar with time-locked contracts, recovery can be complicated. • Unsecured pre-signed transactions can cause security breaches.
Kai Lu, Chongyang Zhang - 2020	• Eliminates the need for MPC third parties. • Efficiency improved with off-chain computation. • Blockchain ensures that results stored are tamper-proof.	• Off-chain computation causes bottlenecks. • Increased storage costs on blockchain. • Scalability issues when the number of players increases.
G Prakash -2019	• Uses MPC with blockchain for improving privacy. • Parallelization betters performance.	• Partial execution. • Complexity in jumbled circuits. • Off-chain execution bottlenecks.
H Gao, Z Ma, et.al. -2019	• Fairness achieved via game-theoretic incentives. • Concept of rewards for good actors and penalties for bad actors.	• Dependency on blockchain causes bottlenecks. • Increased cost in malicious situations.

Author	Key Findings	Limitations
Buchade, A. R. -2017	• Suggests multi-level key splitting and validation. • Dynamic key management algorithms.	• Reliable third generates the keys increasing dependency. • Lacks practical testing. • Issues concerning members of dynamic groups.
D Leung, A Suhl, et.al. -2018	• Sharding decreases the load on node storage. • Quicker bootstrapping for freshly added network nodes.	• Scalability is affected due to sharding under high loads. • Cross-shard delay is caused by dependency on cryptographic proofs.
A Buchade, R Ingle -2016	• Asmuth-Bloom, Shamir's Scheme, and Rabin's IDA were compared. • Rabin's IDA is less secure but has better storage efficiency. • While being efficient, Asmuth-Bloom has high overhead.	• Rabin's IDA is vulnerable to intrusions. • Asmuth-Bloom does not support splitting keys. • Rabin's IDA is not flexible for high-security systems.
AR Buchade, R Ingle -2014	• Compares RC4, Blowfish, TripleDES, and AES for the cloud. • While AES has the strongest security, RC4 is the fastest. • Depending on if cloud is public, private, or hybrid key management differs.	• Data is lost when key is lost. • Single points of failure cause issues in centralized key management. • Regular important updates increase intricacy and lower confidence.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

3.1.1 Assumptions

- **Cooperative Stakeholders:** Users and trusted contacts are responsive in performing key recovery and shard duties.
- **Blockchain Availability:** The network remains active and decentralized with minimal delays in smart contract execution.
- **Threshold Recovery:** Key recovery works via pre-defined shard thresholds.
- **Multi-Signature Usage:** All parties understand and adhere to multi-signature protocols.
- **Time-Lock Enforcement:** Smart contracts enforce withdrawal timing restrictions.
- **Privacy Assurance:** All sensitive data remains encrypted throughout.
- **Fault Recovery:** The system supports secure, fault-tolerant key restoration.

3.1.2 Dependencies

- **Blockchain:** Used for validating transactions and storing public keys.
- **Smart Contracts:** Handle automated withdrawal and time-locks.
- **Public Key Ledger:** Blockchain-based key registry.
- **Key Management API:** Supports key creation, splitting, and recovery.
- **Shard Management:** Relies on trusted contacts to hold shards.
- **Blockchain SDKs:** Libraries (e.g., Web3.js) for smart contract interaction.
- **Platform Compatibility:** Works across web and mobile platforms.

3.2 Functional Requirements

3.2.1 User Interaction Module

Provides a secure web interface for:

- Asset transfers and shard recovery,
- Monitoring withdrawals and approvals,
- Participating in multi-party confirmations.

3.2.2 Key Management Module

Handles:

- Master key generation,
- Shard creation via threshold cryptography,
- Secure shard distribution and key reconstruction.

3.2.3 Transaction Processing Module

Manages:

- Multi-party transaction approvals,
- Transaction signing via MPC or multi-sig,
- Logging metadata and managing disputes/timeouts.

3.2.4 Blockchain Integration Module

Performs:

- Smart contract interactions,
- On-chain storage of keys and transactions,
- Blockchain-based status verification.

3.2.5 Fault Tolerance Recovery Module

Ensures resilience via:

- Threshold-based recovery,
- Social recovery options,
- Time-locks to delay unauthorized access.

3.2.6 Vault Administration Module

Supports:

- Vault and participant management,
- Withdrawal reviews,
- Coordinated recovery efforts.

3.3 Nonfunctional Requirements

3.3.1 Performance Requirements

- User actions should respond within 2 seconds.
- Smart contracts confirmed within blockchain limits (12–15s).
- Key recovery within 5 seconds after quorum.
- System supports 500+ concurrent users.

3.3.2 Safety Requirements

- No permanent storage of sensitive data.
- Recovery mechanisms prevent permanent loss.
- Alerts for abnormal behavior via secure channels.

3.3.3 Security Requirements

- Keys use secure random generation and encryption.
- MFA enforced for sensitive tasks.
- TLS 1.2+ for all communication.
- Audited smart contracts.
- Recovery requires quorum, not a single point.

3.3.4 Software Quality Attributes

- **Reliability:** Minimal downtime with failover support.
- **Maintainability:** Modular architecture for easy updates.
- **Scalability:** Handles increasing users and transactions.
- **Usability:** Clear, intuitive interface.
- **Portability:** Deployable on cloud, on-premise, or hybrid.

3.4 External Interface Requirements

3.4.1 User Interfaces

Secure, responsive UI with:

- **Accessibility:** Responsive design and WCAG 2.1 compliant.
- **Authentication:** OTP/2FA login and personalized dashboard.
- **Transactions:** Panels for initiation, approval, history tracking.
- **Shard Management:** Secure shard handling with status prompts.
- **Admin Tools:** Lifecycle, user, and recovery management.
- **Alerts:** Real-time system feedback and confirmations.
- **Security:** Role-based access, no sensitive data exposed in-browser.

3.4.2 Software Interfaces

Key technologies:

- **Backend:** Python for cryptography and APIs.
- **Smart Contracts:** Solidity for Ethereum logic.
- **Frontend:** React.js for user interface.

3.5 Analysis Models: SDLC Model to be applied

For this project, the Agile Software Development Life Cycle (SDLC) model was adopted. Given the modular nature of the system—which includes key generation and shard distribution using AWS Lambda, a React based frontend, and smart contracts deployed on the blockchain. Agile allowed for the efficient development and integration of each component through iterative sprints. The model enabled continuous development, testing, and refinement, ensuring each module could be validated independently before full integration. This approach also allowed the team to identify and resolve issues early, particularly in cryptographic processes and contract behavior.

Agile's emphasis on adaptability and collaboration was especially beneficial due to the cross-functional requirements of the project. Backend, frontend, and blockchain components were developed in parallel and integrated incrementally. Regular sprint reviews and feedback loops ensured the system remained aligned with the functional and security requirements. The Agile model ultimately facilitated faster development, better risk management, and a more robust, user-focused final product.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

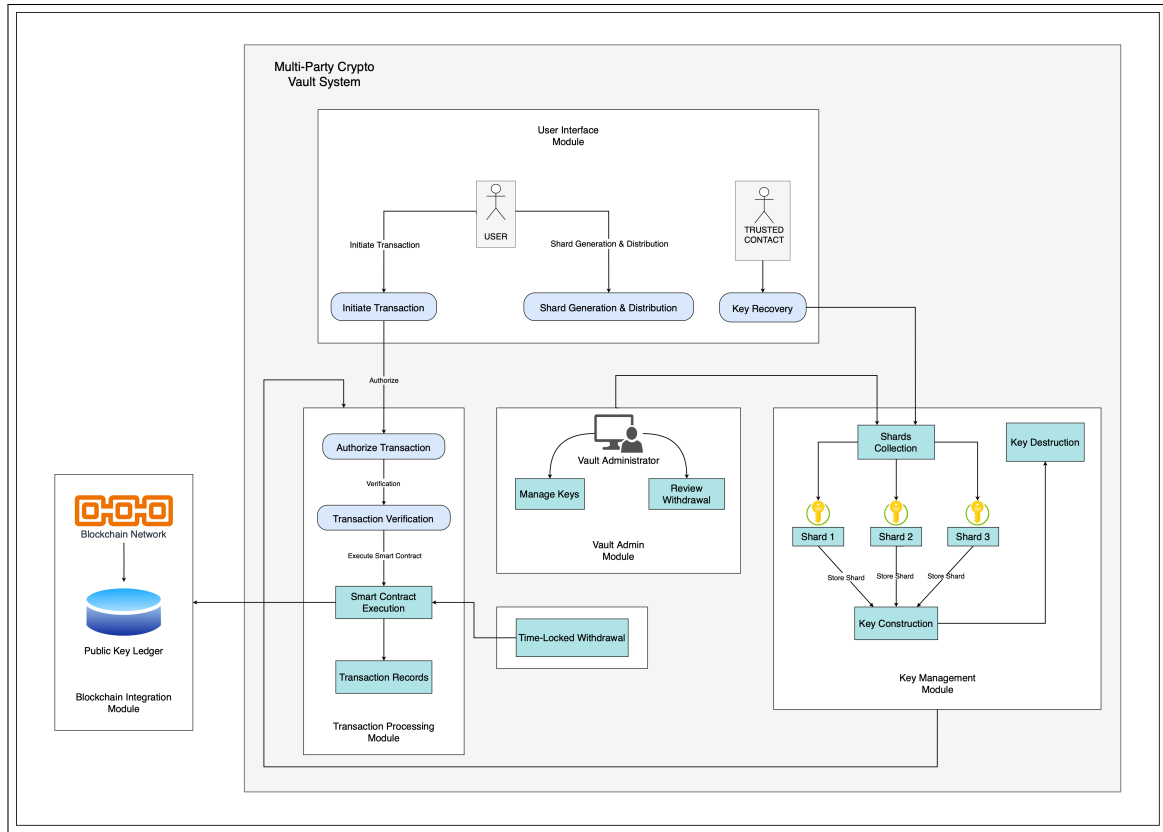


Figure 4.1: System Architecture

The architecture diagram in Fig. 4.1 illustrates the integration of a Multi-Party Vault System for Secure Digital Transactions. The architecture consists of several key components:

- **User Interface:** **Initiate Transaction:** The user (such as a business analyst, tester, or developer) inputs the required transaction details, which are processed and distributed to various system components. Additionally, the user can recover lost key shares through trusted contacts.
- **Multi-Party Vault System:**
 - **Shard Generation & Distribution:** When a transaction is initiated, the system generates cryptographic key shards and distributes them to multiple trusted entities.

- **Shard Collection & Key Construction:** During the withdrawal or completion of a transaction, these shards are collected and combined to reconstruct the original cryptographic key.
- **Key Destruction:** After the key has served its purpose, it is securely destroyed to prevent misuse.
- **Vault Administrator:**
 - **Manage Keys:** The vault administrator oversees the management and integrity of cryptographic keys. They can approve or review key generation, shard management, and withdrawals.
 - **Review Withdrawal:** The administrator ensures all withdrawal actions are authorized and compliant with security policies.
- **Transaction Flow:**
 - **Transaction Authorization:** After the key is reconstructed, the transaction is authorized.
 - **Transaction Verification:** The system verifies the transaction against the blockchain to ensure authenticity and security.
 - **Smart Contract Execution:** The blockchain executes a smart contract, ensuring automation and security in the transaction process.
 - **Transaction Records:** All transaction data is stored and logged in the public key ledger for future reference
- **Blockchain Network:**
 - **Public Key Ledger:** The blockchain network serves as a secure, immutable ledger for recording all transaction events, ensuring transparency and integrity.

This architecture ensures secure digital transactions by utilizing multi-party key management, blockchain technology, and secure verification processes.

4.2 Mathematical Model

4.2.1 Shard Generation Using Shamir's Secret Sharing

Shamir's Secret Sharing (SSS) is a (k, n) threshold scheme that divides a secret s into n shares such that any subset of at least k shares can reconstruct the secret, while fewer than k shares provide no information about s .

To generate the shards, we define a polynomial $f(x)$ of degree $k - 1$:

$$f(x) = s + a_1x + a_2x^2 + \cdots + a_{k-1}x^{k-1} \mod p \quad (4.1)$$

Equation 4.1 defines the random polynomial over $GF(p)$ whose constant term is the secret s and whose coefficients $a_1, \dots, a_{k-1}$ are chosen uniformly at random to ensure unpredictability.

Each participant i receives a share (x_i, y_i) , computed as:

$$y_i = f(x_i) \mod p \quad (4.2)$$

Equation 4.2 shows how each share is generated by evaluating the polynomial at a distinct, nonzero point x_i , producing the point (x_i, y_i) .

These shares are securely distributed to designated key holders without being stored in any centralized repository.

4.2.2 Shard Reconstruction via Lagrange Interpolation

To reconstruct the private key, at least k shares $(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)$ are required. Using Lagrange interpolation, the polynomial $f(x)$ can be reconstructed as:

$$f(x) = \sum_{j=1}^k y_j \prod_{\substack{m=1 \\ m \neq j}}^k \frac{x - x_m}{x_j - x_m} \mod p \quad (4.3)$$

Equation 4.3 reconstructs $f(x)$ via Lagrange basis polynomials, combining the known points (x_j, y_j) so that the resulting polynomial matches all those points exactly.

The secret key s is then extracted as the constant term:

$$s = f(0) \mod p \quad (4.4)$$

Equation 4.4 shows that evaluating the reconstructed polynomial at $x = 0$ yields the original secret s .

4.2.3 Enhanced Multi-Secret Sharing Scheme

While traditional SSS allows sharing a single secret, our system extends this approach using a multi-secret sharing scheme for efficiency and scalability. We adopt an enhanced (t, n) scheme inspired by Pang and Wang (2005), which allows multiple secrets to be shared simultaneously.

- **Polynomial Construction:** A single polynomial $h(x)$ of degree $n + p - 1$ encodes multiple secrets $P_1, P_2, \dots, P_p$ as:

$$h(x) = P_1 + P_2x + \dots + P_px^{p-1} + a_1x^p + \dots + a_{n+p-1}x^{n+p-1} \mod q \quad (4.5)$$

Equation 4.5 builds a high-degree polynomial whose first p coefficients are the p secrets, with the remaining coefficients random to secure the scheme.

- **Share Distribution:** Each participant i receives a pseudo-shadow $f(r, s_i)$ and a computed share:

$$y_i = h(x_i) \mod q \quad (4.6)$$

Equation 4.6 shows that each share is simply the evaluation of $h(x)$ at the point x_i , analogous to the single-secret case.

- **Secret Reconstruction:** At least t participants must combine their pseudo-shadows to recover $h(x)$ using Lagrange interpolation:

$$h(x) = \sum_{j=1}^t y_j \prod_{\substack{m=1 \\ m \neq j}}^t \frac{x - x_m}{x_j - x_m} \mod q \quad (4.7)$$

Equation 4.7 employs Lagrange interpolation in the same way as before, but on the higher-degree polynomial $h(x)$.

The original secrets $P_1, P_2, \dots, P_p$ are then retrieved as:

$$P_i = h(i - 1) \mod q, \quad \forall i \in [1, p] \quad (4.8)$$

Equation 4.8 extracts each secret by evaluating $h(x)$ at $x = i - 1$, recovering the p embedded secrets.

4.2.4 Advantages of the Enhanced Implementation

- **Parallel Secret Sharing:** Multiple secrets can be shared and reconstructed simultaneously, enhancing efficiency for applications requiring large data sharing.
- **Reduced Computational Complexity:** The use of Lagrange interpolation eliminates the need to solve simultaneous equations, simplifying both distribution and reconstruction.

- **Dynamic Flexibility:** The scheme allows dynamic addition of secrets without requiring redistribution of participant shares.
- **Enhanced Security:** Pseudo-shadows ensure that participant secrets remain undisclosed even after multiple reconstructions, adhering to the (t,n) threshold rule.

4.3 Data Flow Diagram

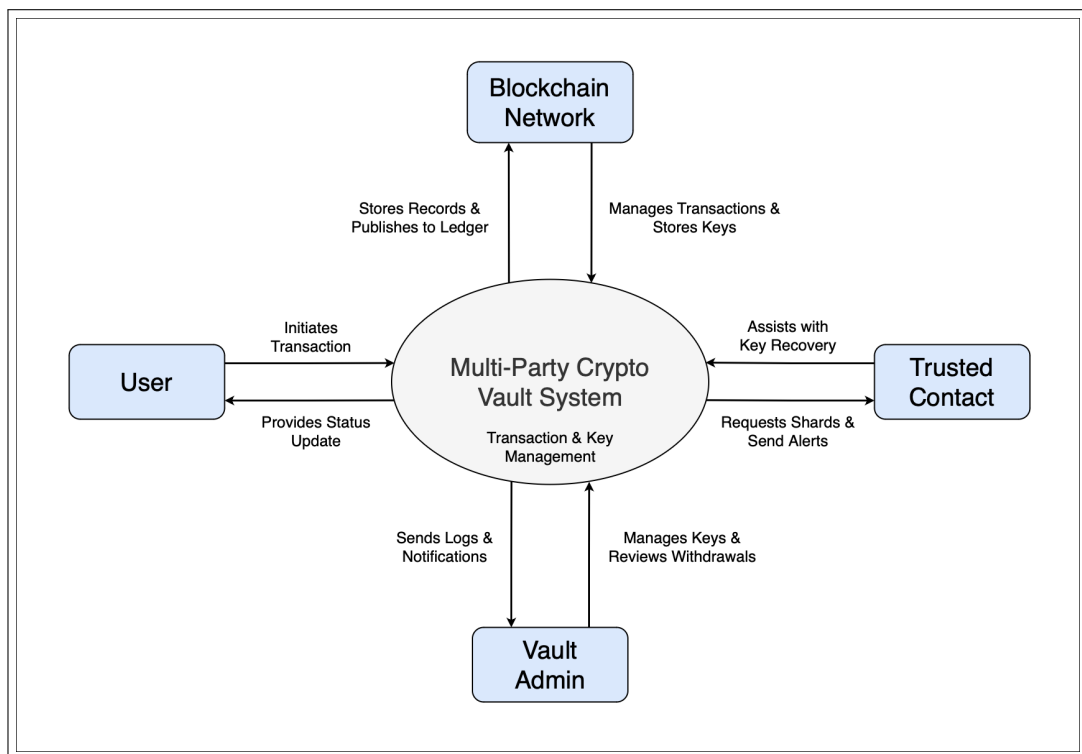


Figure 4.2: Data Flow Diagram - Level 0

The Fig. 4.2 presents a high-level view of the multi-party crypto vault system, outlining the main entities and their core interactions.

- **External Entities:**
 - **User:** Initiates transactions and creates key shards.
 - **Trusted Contact:** Assists with key recovery when needed.
- **Shards Generation & Distribution:** Key shards are created and securely distributed among participants for redundancy and protection.

- **Key Recovery:** Reassembles keys from shards. Trusted Contacts help recover missing shards or authorize reconstruction.
- **Vault Admin:** Oversees key management and transaction authorization, ensuring all security checks are met.
- **Transaction Management:** Processes and submits authorized transactions to the blockchain. Stores them for future reference.
- **Blockchain Network:** Verifies transactions and acts as the immutable public ledger for secure storage.

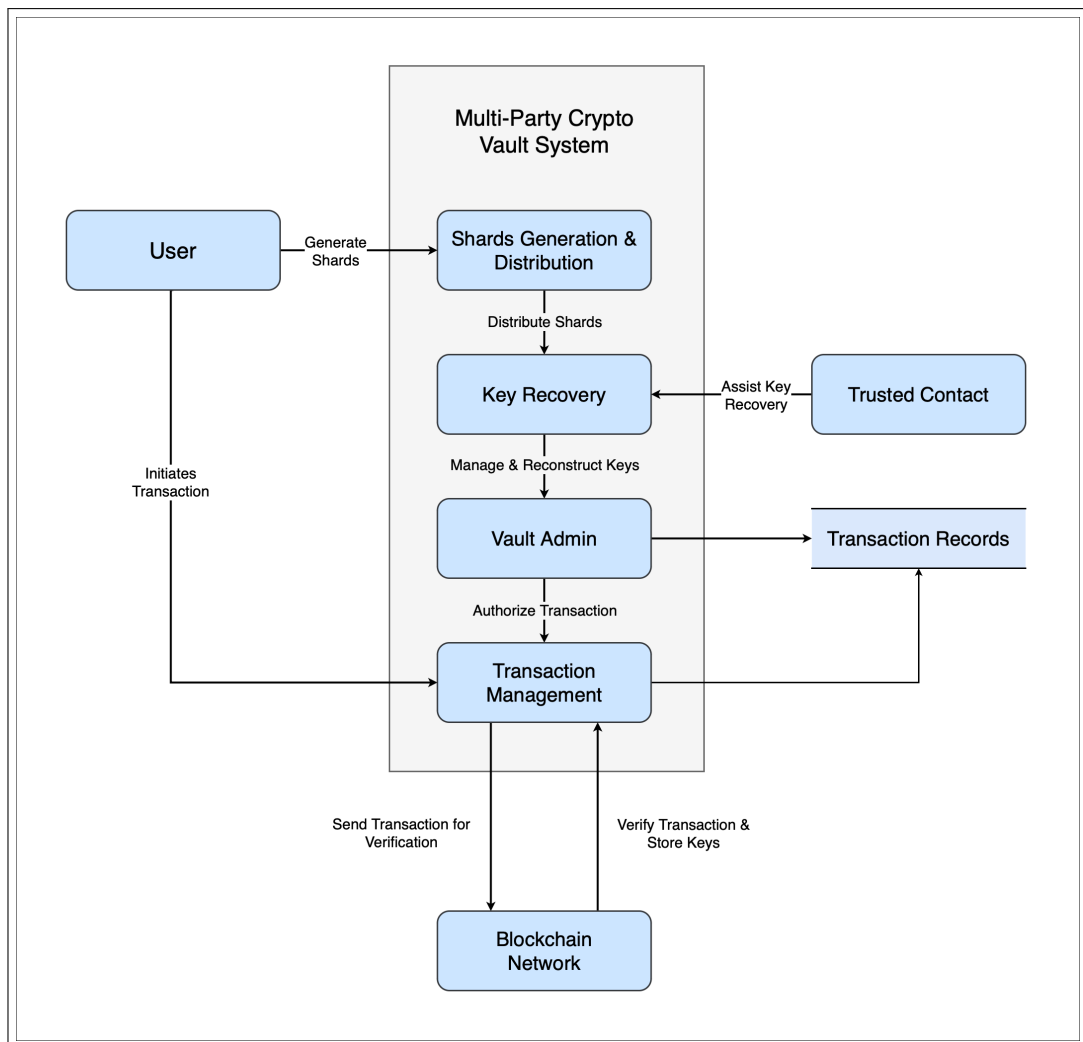


Figure 4.3: Data Flow Diagram - Level 1

The Fig. 4.3 provides a more detailed breakdown of interactions and roles for each component involved in the system, including communication flows.

- **External Entities:**
 - **User:** Initiates transactions and receives updates regarding the transaction status.
 - **Trusted Contact:** Assists with key recovery upon request from the system. May also send alerts or provide shards to facilitate recovery or authorize transactions.
 - **Vault Admin:** Oversees key management and verifies withdrawals or other operations related to keys. Sends notifications or logs to ensure smooth operation and transparency across the system.
- **Blockchain Network:** Manages transaction processing and stores transaction records and keys in the ledger for public transparency. The network ensures that transactions are tamper-proof once verified and published on the blockchain.
- **Feedback Loop:** Users receive status updates about their initiated transactions to ensure transparency. Logs and notifications are sent to relevant stakeholders, such as the vault admin, for monitoring and compliance.

This breakdown highlights the cyclical nature of transaction management within the multi-party crypto vault system. It ensures data integrity through secure storage on the blockchain, distributed key recovery, and ongoing communication between users, trusted contacts, and admins.

4.4 UML Diagram

4.4.1 Use Case Diagram

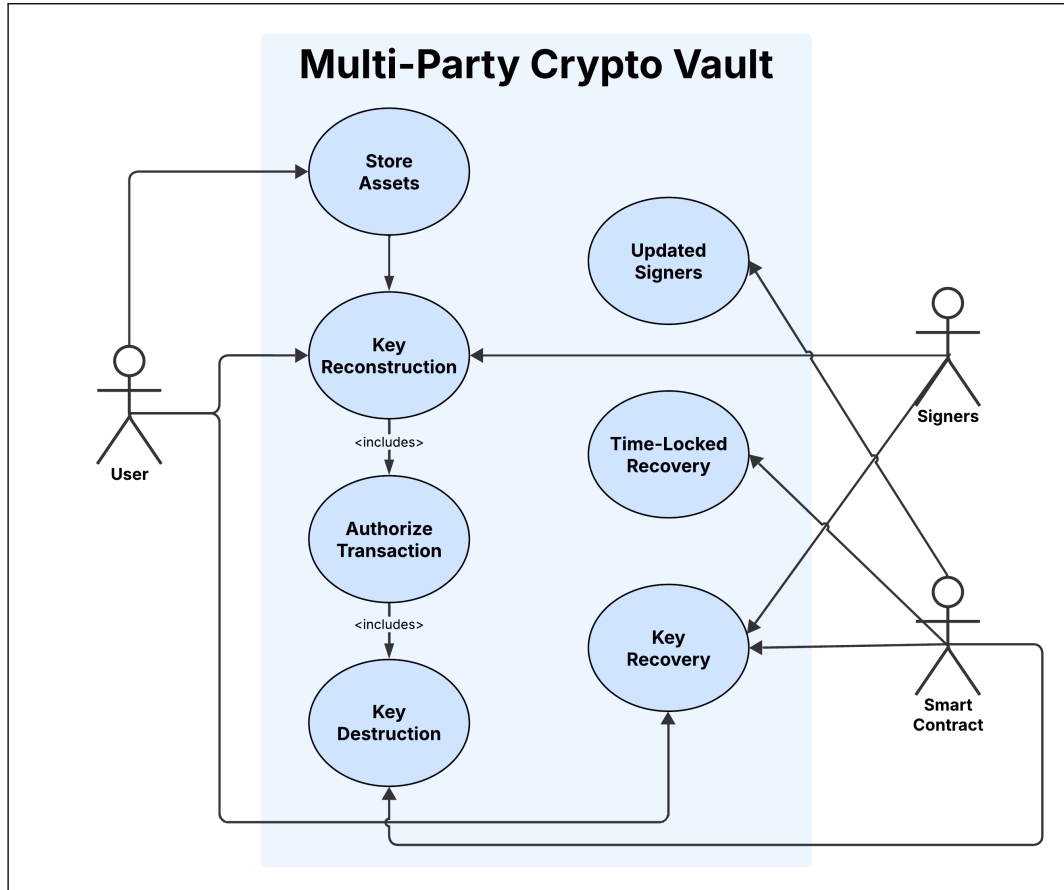


Figure 4.4: Use Case Diagram

The Use Case Diagram in Fig. 4.4 illustrates the core process flow of a Multi-Party Crypto Vault System, highlighting secure asset storage, transaction execution, and key management through coordinated signer actions.

- **Actor:** The actor (e.g., a business analyst or tester) interacts with the system by providing requirements through the `provideRequirements()` method and receiving generated test cases via the `receiveTestCases()` method.
- **StoreAssets:** The user stores digital assets securely in the vault by invoking the `storeAssets()` function. These assets are protected using multi-party cryptographic techniques.

- **KeyReconstruction:** For transaction execution, the cryptographic key is reconstructed by invoking the `keyReconstruction()` method. The key is split into shares, and multiple signers are required to piece the key back together.
- **AuthorizeTransactions:** After the key is reconstructed, the user authorizes transactions by using the `authorizeTransactions()` method. This invokes a smart contract, which enforces rules and automates the process.
- **KeyDestruction:** After a transaction is completed, the key is securely destroyed using the `keyDestruction()` method to prevent future misuse.
- **KeyRecovery:** If the key is lost or the user needs to recover it, the system invokes `keyRecovery()` using a time-locked mechanism. Signers and the smart contract work together to recover the key under defined security conditions.
- **UpdateSigners:** The `updateSigners()` function is used to modify the list of authorized signers. The smart contract ensures that this update complies with the system's policies.
- **Time-Locked Recovery:** The system provides a `timeLockedRecovery()` mechanism, enabling key recovery after a predefined lock period, enhancing security in case of lost or compromised keys.

4.4.2 Activity Diagram

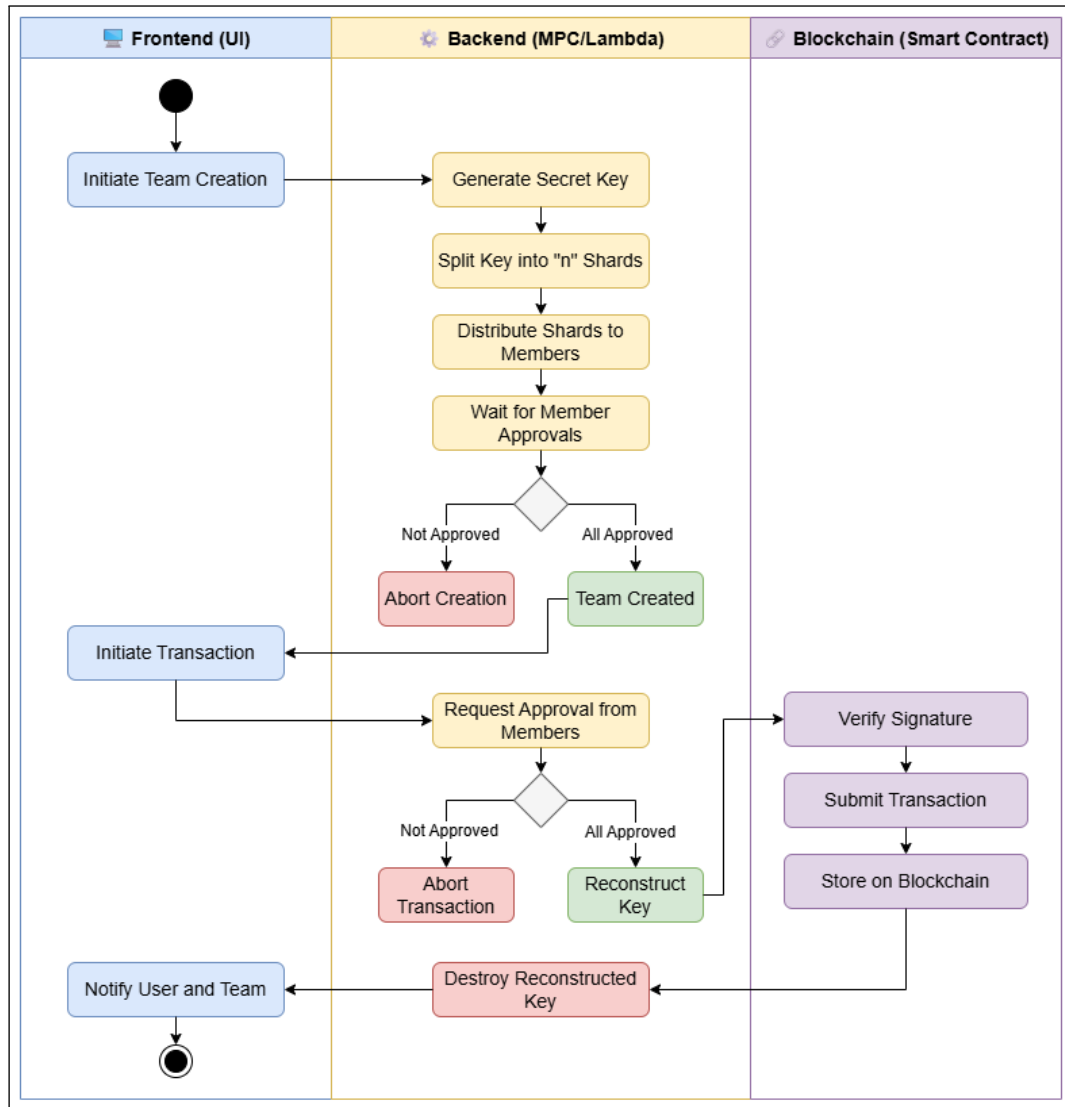


Figure 4.5: Activity Diagram

The Activity Diagram in Fig. 4.5 illustrates the workflow of the Multi-Party Crypto Vault system, detailing the team-based coordination for secure transaction authorization using MPC and smart contracts.

- **Initiate Team Creation:** A user triggers team creation, prompting backend logic to generate a cryptographic secret key.
- **Generate and Distribute Shards:** The key is split into shards using MPC and distributed to team members via email.

- **Member Approvals:** Team members must approve the creation; the process aborts if any member declines.
- **Initiate Transaction:** Any member can initiate a transaction.
- **Request Approval from Members:** Approval requests are sent to all members for transaction consent.
- **Key Reconstruction and Validation:** Upon unanimous approval, shards are recollected to reconstruct the key and generate a transaction signature.
- **Blockchain Submission:** The reconstructed signature is verified and the transaction is submitted to the blockchain.
- **Destroy Key and Notify:** The key is destroyed after the transaction is confirmed, and members are notified.
- **Failure Handling:** If any member withholds approval, the action is aborted and no record is submitted to the blockchain.

4.4.3 Sequence Diagram

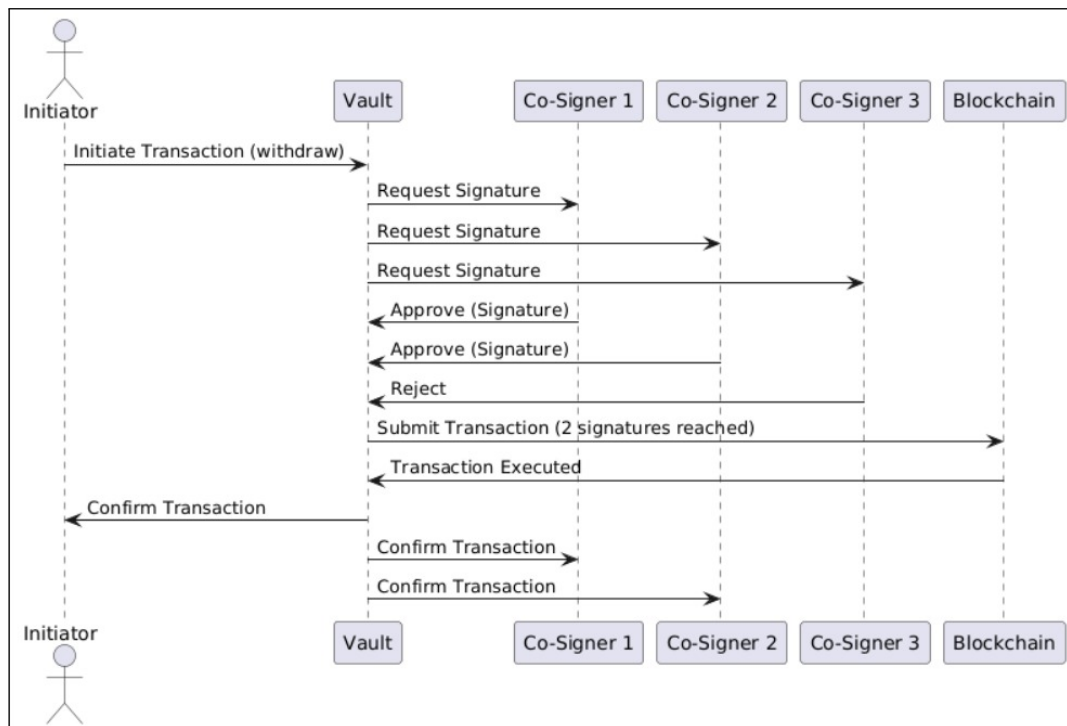


Figure 4.6: Sequence Diagram

The Sequence diagram in Fig. 4.6 represents a multi-party transaction approval process using a vault with multiple co-signers. Here's a breakdown:

- **Initiate Transaction:** The initiator requests a transaction (e.g., withdrawal) from the vault.
- **Request Signature:** The vault sends signature requests to multiple co-signers.
- **Approval or Rejection:** Co-signers can either approve or reject the request. Two approvals are needed to proceed.
- **Submit Transaction:** Once two signatures are collected, the vault submits the transaction to the blockchain for execution.
- **Confirm Transaction:** After execution, the vault confirms the transaction to the initiator and co-signers.

The UML diagrams collectively depict the secure and coordinated workflow of the Multi-Party Crypto Vault system. They capture key processes such as asset storage, key reconstruction, transaction authorization, and key destruction. Interactions among users, backend services, and smart contracts ensure a robust and secure transaction lifecycle.

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

Development Timeframe: 6 months The timeline covers system conceptualization to final deployment, broken down into the following phases:

- **Research Design (1.5 months)**
 - Conduct thorough research on cryptographic techniques like Multi-Sig and MPC.
 - Design a robust system architecture supporting distributed control and fault tolerance.
 - **Key Objectives:**
 - * Evaluate Multi-Sig and MPC techniques for secure asset management.
 - * Develop a blueprint covering the user interface, key management, and back-end.
 - * Identify failure points and incorporate fault tolerance strategies.
- **Front-End Back-End Development (2 months)**
 - Develop the user interface and server-side logic for cryptographic functions, transaction processing, and key management.
 - **Key Objectives:**
 - * Design a user-friendly interface for crypto asset management and transaction approval.
 - * Implement transaction logic, key management, and blockchain interaction.
 - * Seamlessly integrate with the chosen blockchain network.
- **Integration of Cryptographic Methods (1 month)**
 - Integrate Multi-Sig and MPC protocols to enhance secure transaction approval and key management.
 - **Key Objectives:**
 - * Implement Multi-Sig for distributed transaction approval.
 - * Integrate MPC to ensure private transaction authorization without exposing private keys.

- * Add mechanisms like time-locked transactions for participant failure scenarios.
- **Testing Optimization (1 month)**
 - Conduct comprehensive functional and security testing, followed by optimization.
 - **Key Objectives:**
 - * Verify system functionality across key management and blockchain interactions.
 - * Perform security audits to detect and fix vulnerabilities.
 - * Optimize system performance, focusing on reducing latency and enhancing MPC handling.
- **Final Implementation Documentation (0.5 months)**
 - Finalize the full system implementation and create detailed documentation for users and developers.
 - **Key Objectives:**
 - * Integrate all components and prepare the system for deployment.
 - * Develop user manuals and technical guides.
 - * Conduct a final security and readiness review.

5.1.2 Project Resources

1. **Developers:** The team includes a Project Lead to oversee progress, a Front-End Developer for UI design and implementation, a Back-End Developer for server logic and blockchain integration, a Cryptography Specialist for secure methods like Multi-Sig and MPC, and a Tester for functional and security validation.
2. **Software:** The project will use Python, JavaScript, and Solidity for development; PyCryptodome and Web3.js for cryptographic and blockchain functionalities; GitHub/GitLab for version control; and Mocha/Jest for automated testing.
3. **Hardware:** Team members will use laptops/desktops with necessary dev tools. Cloud platforms like AWS or Google Cloud, or a local server, will host and test the prototype for scalability and availability.

5.2 Risk Management

Given the technical and security-focused nature of this project, risk management is crucial for ensuring the project's success. The following sections outline key risks, their analysis, and the proposed mitigation strategies.

5.2.1 Risk Identification

- **Key Compromise Risk:** Risk of private keys being compromised during development or testing.
- **Performance Issues:** MPC may introduce latency and communication overhead, making the system inefficient for real-time transactions.
- **Data Loss:** Failure to manage keys properly can lead to irreversible loss of crypto assets.
- **Cryptographic Vulnerabilities:** Implementation errors in Multi-Sig or MPC could lead to security breaches.
- **Integration Risks:** Difficulty integrating blockchain with the multi-party cryptographic algorithms.

5.2.2 Risk Analysis

- **Key Compromise:** High risk, high impact. Exposure of private keys could lead to asset theft.
- **Performance Issues:** Medium risk, high impact. Latency in MPC computations could hinder usability for certain applications.
- **Data Loss:** High risk, high impact. Loss of keys would result in permanent loss of crypto assets.
- **Cryptographic Vulnerabilities:** Medium risk, high impact. An attack on the cryptographic protocols could compromise the vault.
- **Integration Risks:** Low risk, medium impact. Complexity in integrating blockchain can delay the project but won't compromise security directly.

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Key Compromise Mitigation:** Use hardware security modules (HSMs) for secure key storage and enforce strict access controls. Conduct regular key rotation and enforce multi-factor authentication (MFA).
- **Performance Optimization:** Prioritize optimization of MPC protocols. For high-frequency applications, consider fallback to Multi-Sig for faster processing in non-sensitive cases.
- **Data Loss Prevention:** Implement robust backup and recovery procedures for key shares using split-key management and social recovery models.
- **Cryptographic Security:** Conduct thorough security audits of the cryptographic protocols. Use well-established libraries and algorithms to avoid custom implementations that could introduce vulnerabilities.
- **Integration Strategy:** Plan early-stage blockchain integration with crypto vault features. Use modular architecture to ensure flexibility in integrating different blockchain networks.

5.3 Project Schedule

5.3.1 Project Task Set

- **Requirement Analysis:** Gather system requirements, identify actors, and define system functionalities.
- **System Design:** Develop system architecture, use case diagram, and sequence diagram for process understanding.
- **Modeling:** Create class diagram, activity diagram, and deployment diagram to model system structure, behavior, and deployment.
- **Development:** Implement system modules based on the design and integrate cryptographic functionalities using MPC.
- **Testing:** Perform unit, integration, and system testing to validate functional and security requirements.
- **Deployment and Maintenance:** Deploy the system, monitor performance, and maintain security updates.

5.3.2 Timeline Chart



Figure 5.1: Timeline Chart

5.4 Team Organization

5.4.1 Team structure

- **Rushi Balapure:** Backend Developer, Blockchain developer
- **Prachi Patil:** Designer, Frontend Developer, DevOps
- **Sameer Patil:** Full Stack Developer, DevOps
- **Samiksha Sarode:** Blockchain developer, Frontend Developer
- **Dr. Shital Girme:** Internal mentor and guide.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The system is modularly designed to enable secure, decentralized, and fault-tolerant vault operations over blockchain using cryptographic key management and smart contracts. Each module performs a distinct set of responsibilities as outlined below:

- **User Interaction Module:** Provides a user-friendly interface for initiating transactions, distributing/recovering key shards, and monitoring vault activity. This module acts as the primary access layer for users.
- **Key Management Module:** Responsible for generating cryptographic key pairs, splitting private keys using threshold-based secret sharing (e.g. Shamir's Secret Sharing) and orchestrating secure distribution and reconstruction.
- **Transaction Processing Module:** Handles transaction lifecycle from validation to multi-party authorization ensuring that only valid, threshold authorized transactions are executed.
- **Blockchain Integration Module:** Connects the system to the blockchain network. Facilitates smart contract deployment, on-chain verification, and transaction submission with immutability guarantees.
- **Fault Tolerance and Recovery Module:** Enables secure key recovery in cases of key loss by recollecting a threshold number of shards from authorized users or trusted contacts.
- **Vault Administration Module:** Allows designated administrators to manage key lifecycles, perform monitoring and approval of transactions, and assist in emergency recovery operations.

6.2 Tools and Technologies Used

The implementation leverages a wide range of tools, platforms, and technologies to support secure multi-party computation, blockchain integration, and user interface development:

- **Programming Languages:**
 - **Python:** For backend development and implementation of cryptographic operations.

- **Solidity:** For smart contract development on Ethereum-compatible blockchains.
- **JavaScript/TypeScript:** For frontend interactions and web3 integrations.
- **Blockchain Tools:**
 - **Ethereum/Polygon:** Blockchain networks used for smart contract deployment.
 - **Web3.js or Ethers.js:** For client-side blockchain interaction.
 - **Truffle/Hardhat:** For compiling, testing, and deploying smart contracts.
- **Cryptographic Libraries:**
 - **OpenSSL / NaCl / PyCryptodome:** Used for secure encryption, decryption, and key splitting mechanisms.
- **Frontend Frameworks:**
 - **React.js / Vue.js:** For building responsive and interactive user interfaces.
- **Cloud/DevOps Tools (Optional):**
 - **AWS/GCP/Azure:** Cloud hosting and scaling of backend services.
 - **Docker & GitHub Actions:** For containerization and CI/CD pipeline automation.

6.3 Algorithm Details

6.3.1 Transaction Processing Algorithm

1. Input: Transaction request from user.
2. Verify user identity and permissions using authentication.
3. Request key shards from multiple signers.
4. If quorum of key shards is reached:
 - Reconstruct the cryptographic key

- Verify transaction details with the blockchain ledger
 - Sign the transaction using the reconstructed key
 - Submit transaction to the blockchain network for validation
 - If validation is successful, update the transaction ledger
 - Destroy the reconstructed key securely
5. Return confirmation to the user

6.3.2 User Interaction Algorithm

1. Input: User request (transaction, key recovery, view balance)
2. Authenticate user with multi-factor authentication (MFA)
3. If transaction is requested: Forward request to Transaction Processing module
4. If key recovery is requested: Forward request to Vault Administration module
5. Return the result to the user (e.g., balance, transaction status)

6.3.3 Key Management Algorithm

1. Input: Request for key management (generation, reconstruction, destruction)
2. If key generation is requested:
 - Generate cryptographic key
 - Split key into multiple shards using Shamir's Secret Sharing
 - Distribute shards to signers
3. If key reconstruction is requested:
 - Collect key shards from signers
 - Reconstruct the key if quorum of shards is met
4. If key destruction is requested:
 - Securely delete the reconstructed key
5. Return confirmation of the action to Vault Administration

6.3.4 Blockchain Network Algorithm

1. Input: Signed transaction from Transaction Processing module
2. Broadcast the transaction to the blockchain network
3. Wait for the transaction to be added to a block and confirmed
4. If confirmation is successful: Update the public ledger with the new transaction
5. Return confirmation or error to Transaction Processing module

6.3.5 Vault Administration Algorithm

1. Input: Administration request (key recovery, update signers)
2. If key recovery is requested:
 - Initiate time-locked recovery process
 - Collect key shards from trusted signers
 - Reconstruct the key after time-lock expires
 - Forward key to the Key Management module
3. If signers update is requested:
 - Authenticate the administrator
 - Add/remove signers based on request
 - Update the list of authorized signers in t

CHAPTER 7

SOFTWARE TESTING

7.1 Types of Testing

Testing plays a critical role in ensuring the correctness, performance, security, and usability of the system. The testing process followed for this project can be broadly classified as follows:

7.1.1 Based on Testing Approach

- **Black Box Testing:** Focuses on the system's external behavior. Testers provide inputs and examine outputs without any knowledge of the internal code structure. For example, testing whether the vault unlocks only when the correct number of shard approvals are collected.
- **White Box Testing:** Involves testing the internal logic, control flow, and data flow of modules like shard generation and smart contract functions. Developers validate if the implementation adheres to logical correctness.
- **Gray Box Testing:** Combines both white-box and black-box testing techniques. It is particularly helpful in integration testing and testing smart contracts, where both internal logic and external interaction matter.

7.1.2 Based on Development Phase

- **Alpha Testing:** Conducted by the developers internally to identify bugs and usability issues before releasing the system to external users.
- **Beta Testing:** Performed by a limited group of actual users to provide feedback on real-world usage, uncover edge-case bugs, and test system behavior under real conditions.

7.1.3 Functional & Non-Functional Testing

7.1.3.1 Functional Testing

This category of testing validates the system against its functional requirements:

- **Unit Testing:** Tests individual functions like key generation, shard splitting, etc.

- **Integration Testing:** Ensures different modules work together, e.g., verifying smart contract behavior when called by the transaction module.
- **System Testing:** Validates the system as a whole, checking if shard reconstruction leads to successful decryption and withdrawal.
- **Regression Testing:** Ensures that system updates or patches do not break existing functionality.
- **Sanity Testing:** Quick tests after minor fixes to ensure that functionalities are working before deep testing begins.
- **User Interface (UI) Testing:** Validates the responsiveness, design consistency, and functional integrity of the frontend UI.

7.1.3.2 Non-Functional Testing

This type of testing ensures the quality attributes of the system:

- **Performance Testing:** Measures response times of shard reconstruction and blockchain interaction under load.
- **Security Testing:** Evaluates the system's resistance to attacks, data breaches, and unauthorized shard access.
- **Usability Testing:** Ensures the system is intuitive and easy to use by both technical and non-technical users.
- **Reliability and Fault Tolerance Testing:** Simulates failures (e.g., lost shards) and verifies that the system can still function using threshold recovery.
- **Scalability Testing:** Tests the system's ability to scale with an increasing number of users or vaults.
- **Compatibility Testing:** Ensures the web interface works across browsers and the system integrates well with different blockchain networks.

7.2 Test Cases and Test Results

The following test cases were designed to validate the functionality of critical components of the system. Each test includes input actions, expected results, and actual results (as observed during testing).

Table 7.1: Test Cases and Expected Results

Test Case ID	Description	Expected Result	Actual Result
TC-01	Validate input fields for number of owners	Should not accept negative numbers or empty values	Pass
TC-02	Test UI behavior when owner count is large (e.g., 100 owners)	UI must not crash; should dynamically adjust	Pass
TC-03	Validate that 'Create Vault' button remains disabled until mandatory inputs are filled	Button disabled until form is valid	Pass
TC-04	Simulate slow network; test if loading states are handled properly	Proper spinners / loading indicators shown	Pass
TC-05	Verify form validation error messages appear correctly	Validation messages shown below fields	Pass
TC-06	Test input tampering (inspect element) and submit invalid values	Frontend must re-validate before sending	Pass
TC-07	Hit API Gateway directly with invalid payload	Must return HTTP 400 or validation error	Pass
TC-08	Test multiple concurrent vault creation requests	Each request should be processed independently	Pass
TC-09	Simulate API Gateway timeout by artificially delaying Lambda	Gateway should respond with 504 Gateway Timeout	Pass

TC-10	Unauthorized access attempt without correct API key/token	Should return 403 Forbidden	Pass
TC-11	Test Lambda cold start behavior with first request	Acceptable cold start latency; no crashes	Pass
TC-12	Validate API rate limiting and throttling at API Gateway	Exceeding limits should return 429 Too Many Requests	Pass
TC-13	Validate private key generation is random for every vault creation	No two vaults have identical keys	Pass
TC-14	Check shard distribution correctness (number of shards == number of owners)	All shards created and mapped correctly	Pass
TC-15	Simulate partial shard collection (only some owners respond)	Key reconstruction must fail	Pass
TC-16	Reconstruct key with wrong shards	Must not reconstruct; fail securely	Pass
TC-17	Simulate simultaneous shard recollection from multiple vaults	No shard collision; isolation is maintained	Pass
TC-18	Simulate missing shards during vault creation	Key reconstruction fails if required shards are missing.	Pass
TC-19	Test shard distribution with changing owner count	Shards are added/removed correctly without data loss.	Pass

TC-20	Simulate corrupted shards during reconstruction	Key reconstruction fails and detects corrupted shards.	Pass
TC-21	Test key reconstruction under high network latency	System must retry / handle gracefully	Pass
TC-22	Send transaction to blockchain after vault reconstruction	Transaction should be signed and submitted	Pass
TC-23	Simulate blockchain network congestion (high gas fees)	Transaction retries / error displayed	Pass
TC-24	Validate transaction signature against reconstructed private key	Valid signatures; match smart contract expectations	Pass
TC-25	Send malformed transaction data to smart contract	Smart contract must reject invalid inputs	Pass
TC-26	Test with blockchain node outage	System should report failure clearly	Pass
TC-27	Test if a transaction can be reverted if an error occurs.	Transaction fails gracefully with an error if not revertible.	Pass
TC-28	Send a large transaction with a low gas limit.	Transaction fails with a "insufficient gas" error.	Pass
TC-29	Test transaction across different blockchains.	Transaction fails with an error if incompatible.	Pass
TC-30	Test DynamoDB resilience (simulate table deletion)	System should detect and raise fatal errors	Pass

TC-31	Check IAM role permissions for Lambda execution	Least privilege principle maintained	Pass
TC-32	Simulate shard interception during transmission	Data must be encrypted end-to-end	Pass
TC-33	Perform penetration test on API Gateway endpoints	No open ports or vulnerable APIs	Pass
TC-34	Test session hijacking on frontend (cookie/session manipulation)	Sessions must expire / regenerate	Pass
TC-35	Try brute-force reconstructing a key without shards	Practically infeasible; proper MPC logic maintained	Pass
TC-36	Inject SQL-style attacks in input forms	Inputs sanitized; no backend compromise	Pass
TC-37	Rotate API Gateway keys/secrets and test app behavior	App should fail gracefully until keys are updated	Pass
TC-38	Check for replay attack vulnerability on transaction requests	Use of nonce/timestamps to prevent replay	Pass
TC-39	Simulate AWS region outage	System degrades gracefully or reroutes	Pass
TC-40	Full end-to-end integration test (Vault creation → Key sharing → Blockchain transaction)	Success from start to finish	Pass

CHAPTER 8

RESULTS

8.1 Outcomes

1. Successfully developed a decentralized multi-party crypto vault system using MPC and blockchain smart contracts.
2. Achieved secure key distribution and fault-tolerant key recovery mechanisms without relying on centralized storage.
3. Deployed smart contracts for decentralized authorization and transaction validation.
4. Developed a simple frontend interface for user interaction with the vault system.
5. Verified system reliability and security through functional testing and feature validation.

8.2 Results and Outputs

8.2.1 Frontend

The following figures illustrate key user interface components and system workflows of the proposed system:

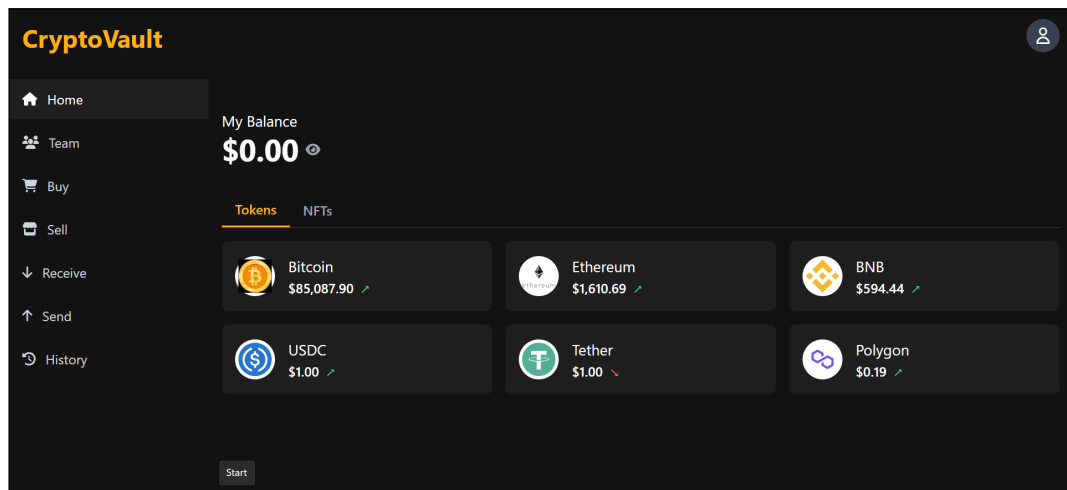


Figure 8.1: Home Page

Home Page (Fig. 8.1): Consists of a user dashboard displaying the current values of various digital tokens. It includes a sidebar for easy navigation and quick access to user profile option.

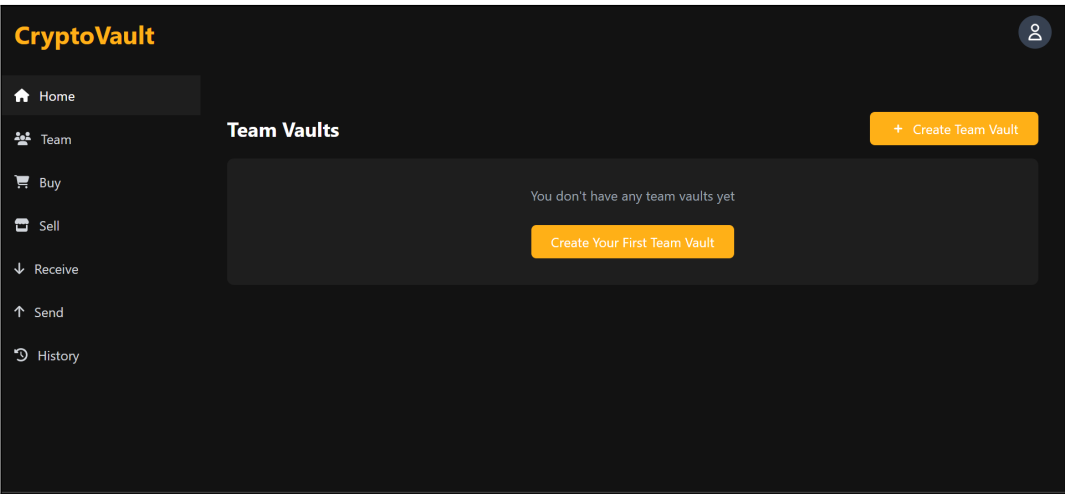


Figure 8.2: Team Page

Team Page (Fig. 8.2): This page provides options to create a new team and manage existing teams. Users can create a team by specifying a team name count of members and their emails. Additionally, they can manage team settings, such as adding/removing members, viewing team details, and deleting the team (subject to conditions like empty assets).

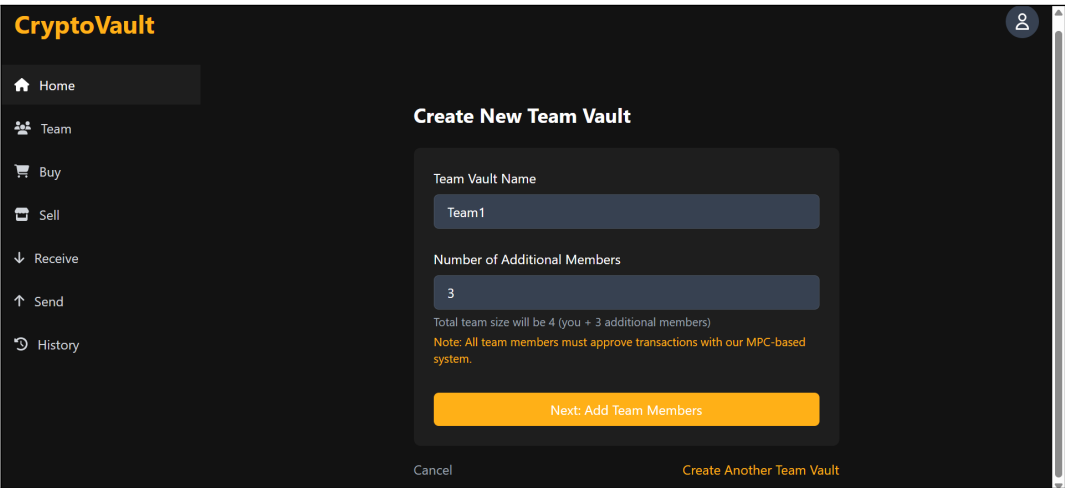


Figure 8.3: Team1: Vault Creation

Create New Team Vault (Fig. 8.3): The option allows users to create a new team vault by specifying the team name (e.g., "Team1") and the count of teammates (excluding the user creating the team).

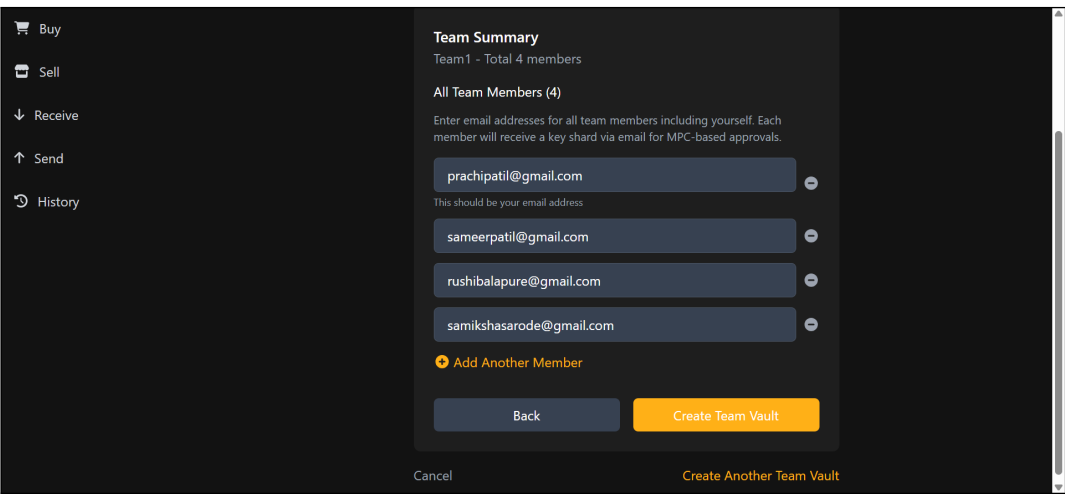


Figure 8.4: Team1: Email Input

Vault Creation & Key Distribution (Fig. 8.4): Users provide emails for themselves and teammates, and upon team creation, a key is generated in backend and shards distributed to the provided emails.

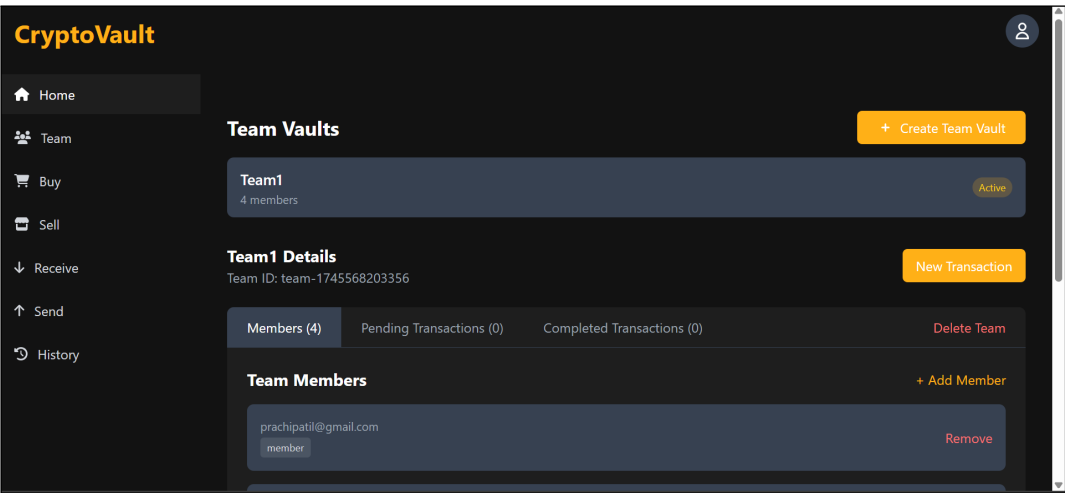


Figure 8.5: Team1: Details - 1

Team1 Details - 1 (Fig. 8.5): Displays the team information, including the team name and its status (active or inactive). It also includes options to add or remove team members, which triggers shard recollection and generates new shards for the updated team. Additionally, there is an option to delete the team, which is only permitted if all assets are empty.

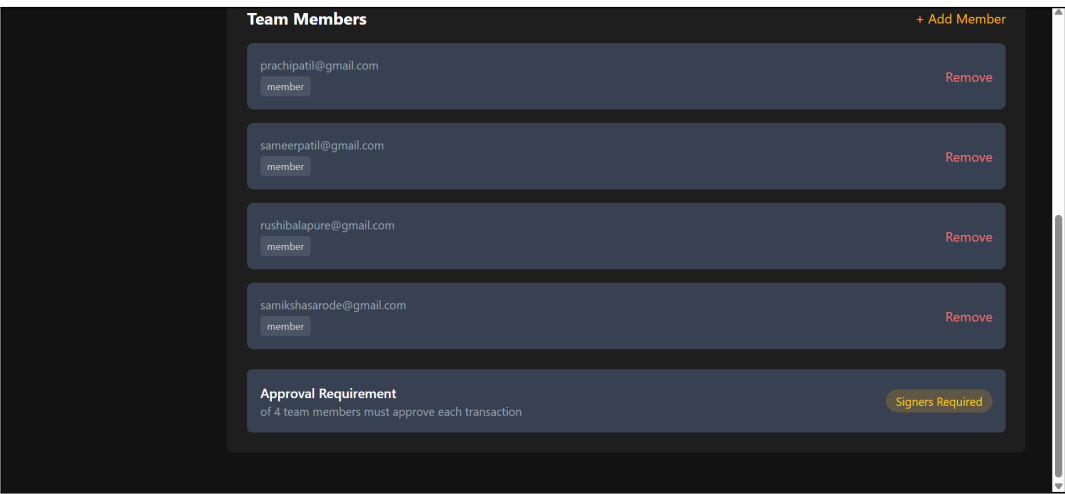


Figure 8.6: Team1: Details - 2

Team1 Details - 2 (Fig. 8.6): Shows a detailed list of team members along with their respective details.

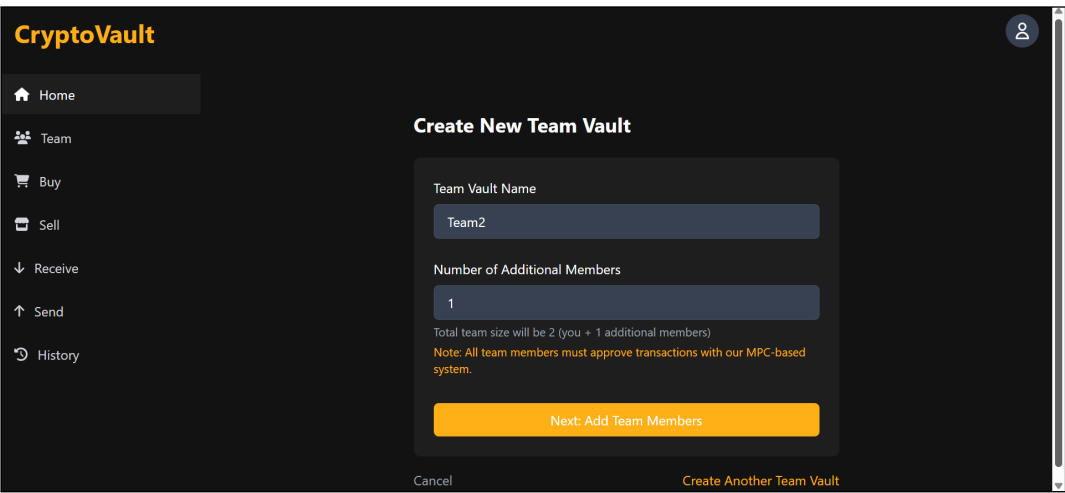


Figure 8.7: Team2: Vault Creation

Create New Team Vault (Fig. 8.7): The option allows users to create a new team vault by specifying the team name (e.g., "Team2") and the count of teammates (excluding the user creating the team).

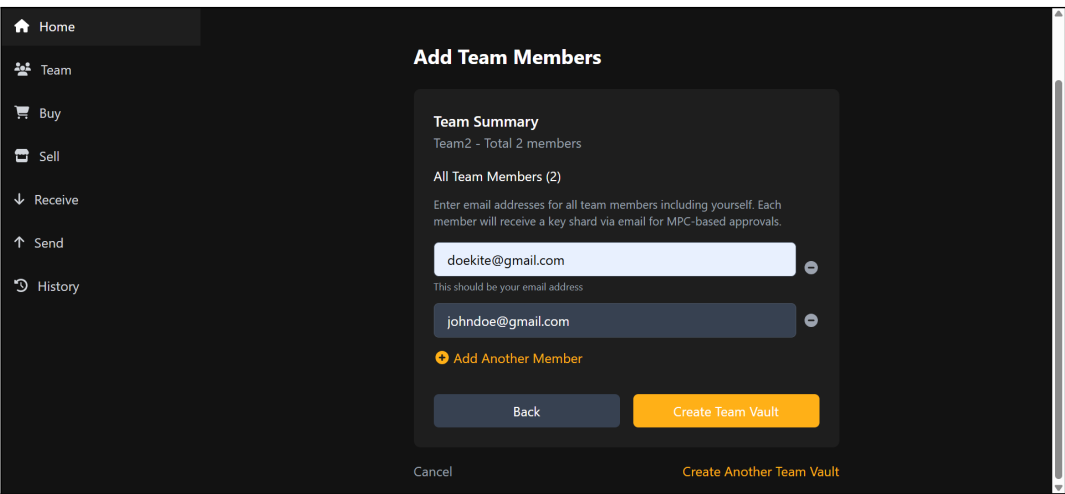


Figure 8.8: Team2: Email Input

Vault Creation & Key Distribution (Fig. 8.8): Users provide emails for themselves and teammates, and upon team creation, a key is generated in backend and shards distributed to the provided emails.

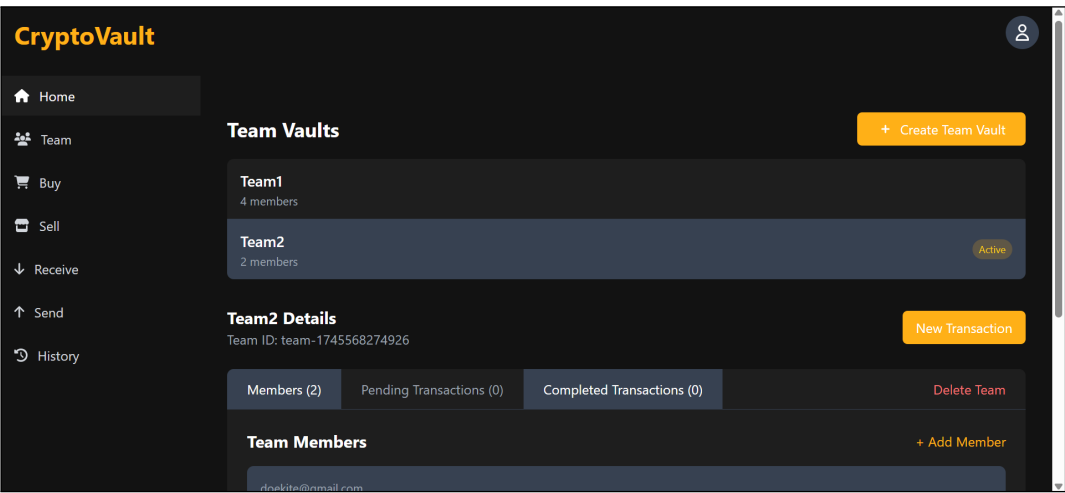


Figure 8.9: Team2: Details - 1

Team2 Details - 1 (Fig. 8.9): If multiple teams are present, the user can select the active team. Only the details of the selected team will be displayed, while other teams will be marked as inactive. Selected "Team2" so marked "active" and shows details of respective team.

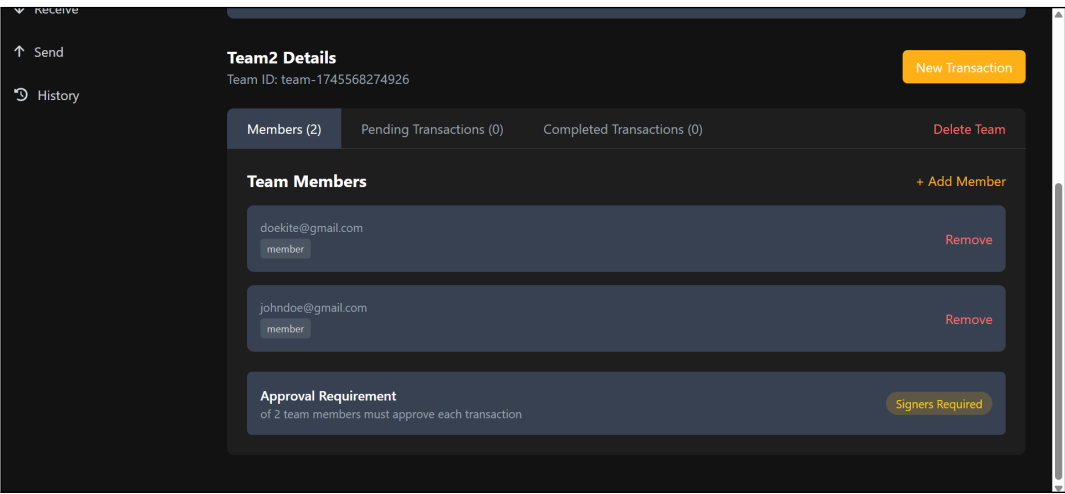


Figure 8.10: Team2: Details - 2

Team2 Details - 2 (Fig. 8.10): Shows a detailed list of team members along with their respective details.

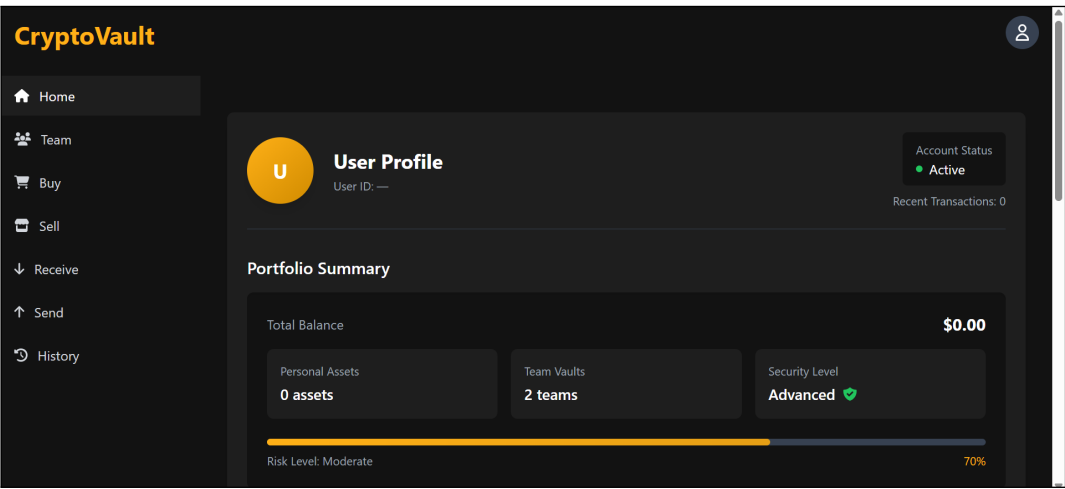


Figure 8.11: User Profile - 1

User Profile -1 (Fig. 8.11): User Portfolio Summary, it displays an overview of the user’s total assets, personal assets, and the team vaults the user is part of, along with the security level of each vault.

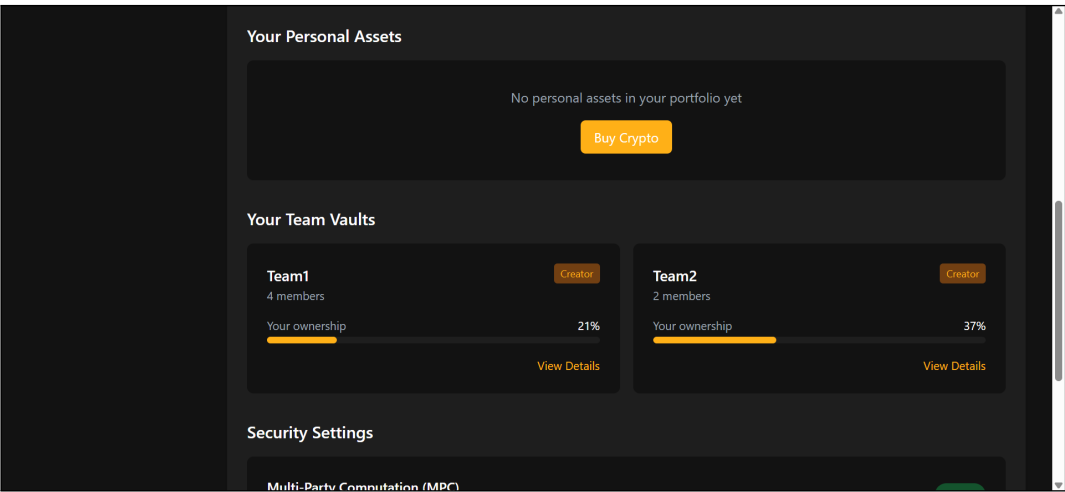


Figure 8.12: User Profile - 2

User Profile - 2 (Fig. 8.12): Personal Assets & Team Vaults, it provides detailed information about the user’s personal assets, the team vaults they are part of, and the respective assets in those vaults.

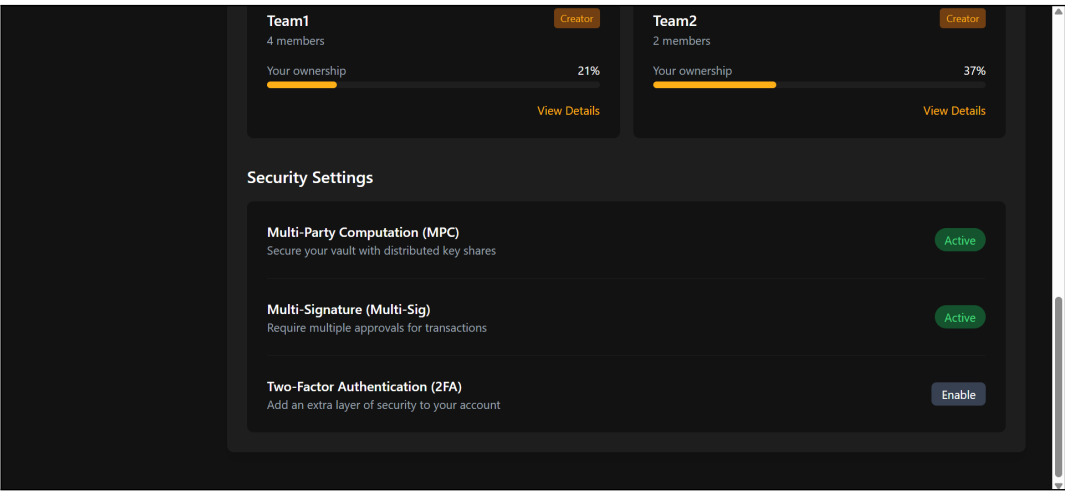


Figure 8.13: User Profile - 3

User Profile - 3 (Fig. 8.13): Security Settings, it shows the user’s security settings, including options to manage authentication and access controls.

Figure 8.14: Buy Assets

Buy Page (Fig. 8.14): Allows the user to select a team, available assets (ETH or Bitcoin), amount, and payment method. Upon clicking "Buy," a trigger is sent to all team members for authorization using shard recollection. The backend regenerates the key matches it actual key, if matched then proceeds with the buy operation or else the operation is failed.

Figure 8.15: Sell Assets

Sell Page (Fig. 8.15): Similar to the Buy Page, this page allows the selection of team name, assets (ETH or Bitcoin), amount to sell, and payout method. Authorization is required from all team members in the backend for the transaction to proceed.

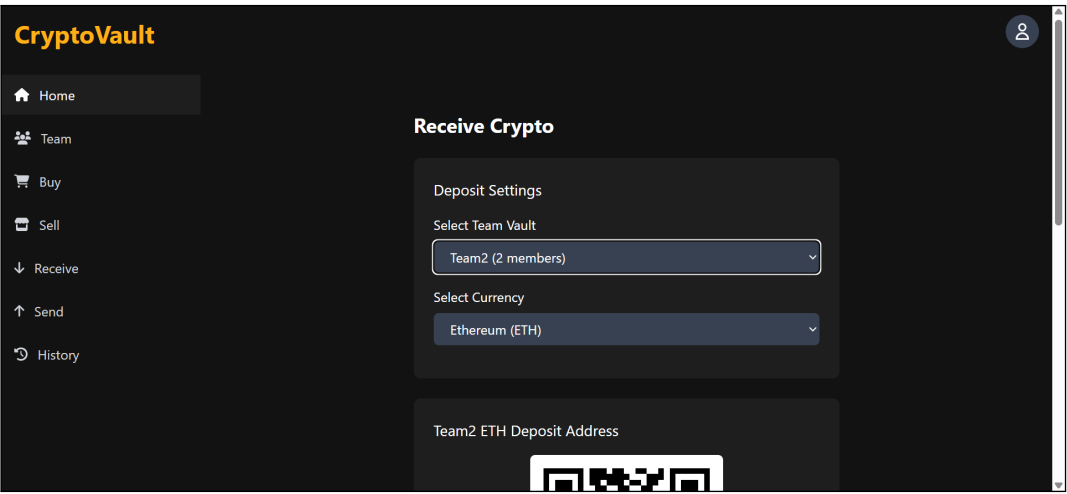


Figure 8.16: Receive Assets - 1

Receive Page (Fig. 8.16): Users can select the team for which assets are to be received, choose the asset type (ETH or Bitcoin), and view a QR code and vault address for receiving the assets.

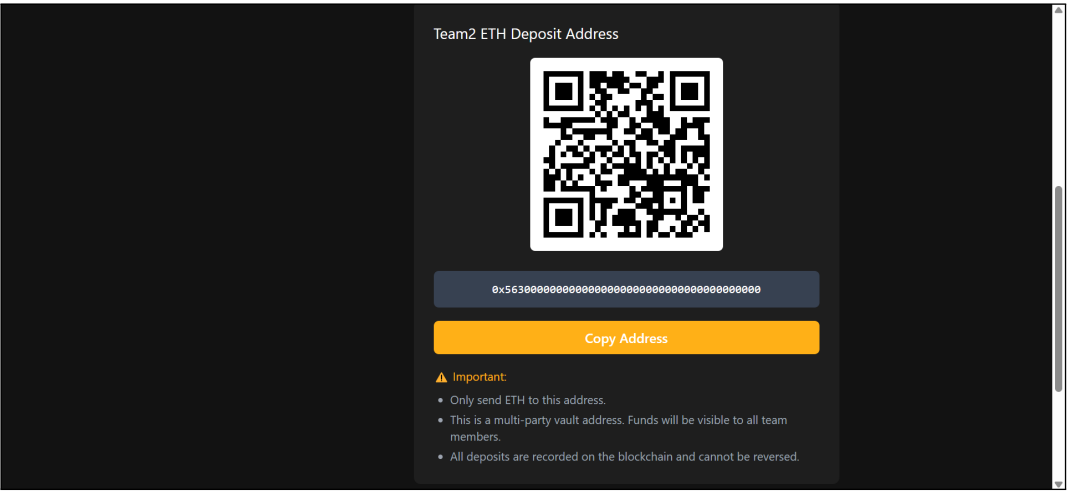


Figure 8.17: Receive Assets - 2

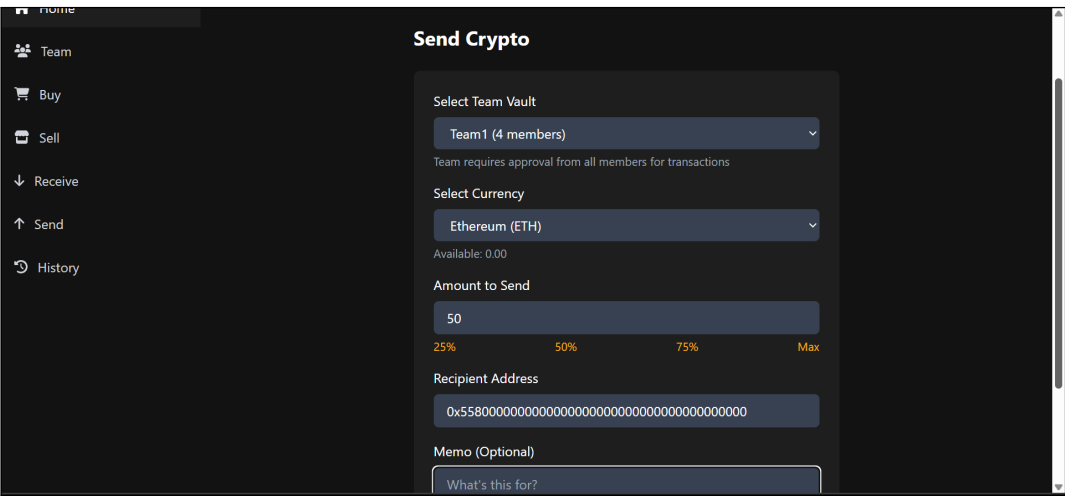


Figure 8.18: Send Assets

Send Page (Fig. 8.18): Allows the user to select a team, asset type, amount, and the vault address to send assets to. Authorization from all team members is required in the backend for the transaction to proceed.

8.3 Results Analysis

8.3.1 Comparative Analysis

Table 8.1: Comparison with Prior Works

Paper & Year	Scheme	Security Model	Latency (ms)	Storage	Key Risk
Vault: Fast bootstrap (2018) [1]	Single-party Bootstrap	Centralized	300	High	Moderate
Efficient multi-sig wallet w/ bloom filter (2021) [2]	n -of- n Multisig	Partial Distributed	220	Medium	Low
BFR-MPC (2019) [4]	MPC + Fairness	Distributed MPC	270	High	Very Low
Threshold multisignature (2020) [10]	Threshold Multisig	Decentralized	210	Low	Very Low
This Work (2025)	Shamir-based Shards	Partially Decentralized	180	Minimal	Negligible

Table 8.1 shows comparative analysis of previously developed systems and our designed system.

CHAPTER 9

CONCLUSION

The successful development and implementation of a secure, fault-tolerant, and decentralized multi-party vault system mark a significant step toward enhancing trust and transparency in digital asset management. The project addressed the complex challenges of key distribution, recovery, and transaction authorization in a collaborative environment without compromising security or usability. The integration of blockchain smart contracts, cryptographic libraries, and cloud-based infrastructure laid a robust foundation for the system's functionality and real-world relevance.

Moreover, the system's architecture promotes accountability and resilience by ensuring that no single point of failure can jeopardize the security of digital assets. The smart contract-driven authorization mechanism further adds auditability and transparency, making it suitable for collaborative or institutional use cases. Scalability and modularity have also been considered, allowing the system to adapt to varying organizational needs. Overall, this solution contributes meaningfully to the evolving landscape of secure decentralized financial systems.

9.1 Applications

1. **Personal Cryptocurrency Wallets:** Multi-party crypto vaults offer enhanced security for individual cryptocurrency users by distributing key ownership across multiple trusted parties. This prevents any single point of failure and mitigates the risk of unauthorized access or loss due to compromised credentials. Users benefit from secure transactions while maintaining full control of their assets through shared key management protocols.
2. **Collaborative Fund Management:** In scenarios involving joint management of funds, such as investment groups or business partnerships, multi-party crypto vaults facilitate secure, shared control over digital assets. Stakeholders can collectively authorize transactions using cryptographic techniques like multi-signature or MPC (Multi-Party Computation), ensuring that funds are only accessed when the required consensus is met.
3. **Decentralized Finance (DeFi) Platforms:** Multi-party vault systems are crucial in safeguarding digital assets within DeFi applications. By decentralizing control over sensitive keys and integrating multi-signature mechanisms, these vaults provide an additional layer of protection, reducing the risk of malicious attacks or unauthorized withdrawals in decentralized finance environments.

4. **Institutional Custody Solutions:** For financial institutions managing large volumes of digital assets, multi-party crypto vaults provide a secure and compliant custody solution. They enable the secure storage and management of digital assets through distributed key management, preventing single points of failure and ensuring regulatory compliance for institutional investors and custodians.

9.2 Advantages

1. **Enhanced Security:** Multi-party crypto vaults provide heightened security by distributing the cryptographic key among multiple trusted parties or devices. This reduces the risk of a single point of failure, as no individual party has full control over the key, preventing unauthorized access to digital assets.
2. **Protection Against Insider Threats:**

Since multiple parties are required to reconstruct the key and authorize transactions, the risk of insider threats is minimized. Collusion among multiple parties would be necessary to compromise the vault, adding an additional layer of protection.
3. **Resilience to Key Loss:** In the case of key loss or compromise by one participant, the vault system can still function securely with the remaining key shards. Additionally, recovery mechanisms, such as time-locked recovery or trusted contacts, allow users to restore access without compromising the system's integrity.
4. **Regulatory Compliance:** Multi-party vault systems are particularly useful for institutions needing to comply with regulatory requirements regarding fund control, transparency, and accountability. By distributing control, these systems ensure that no single entity has unchecked access to funds, aligning with compliance frameworks.

9.3 Future Scope and Recommendations

While the system achieves its core objectives, there is considerable scope for enhancement and scalability:

- Future versions can integrate biometric or two-factor authentication for more granular access control.

- A web-based dashboard can be developed to visualize ongoing transactions, share approvals, and key reconstruction status in real-time.
- The system could be made blockchain-agnostic by adding support for multiple platforms like Hyperledger, Polygon, or private Ethereum networks for industry-specific use cases.
- Smart contracts can be extended to include auditing logs, fallback mechanisms, and gas optimization strategies to improve performance and cost-efficiency.
- Additionally, machine learning models could be integrated for anomaly detection to identify suspicious transaction behavior and alert stakeholders in real-time.

9.4 Technical Observations

The technical journey of the project involved a wide spectrum of tools and platforms that were integrated cohesively to achieve the desired system goals:

- The backend was developed using Python and Node.js, which provided a flexible and efficient environment for building core logic, cryptographic operations, and transaction processing.
- Smart contracts were designed using Solidity to ensure secure, tamper-proof, and decentralized governance over transaction flows and key approvals on the Ethereum blockchain.
- AWS Lambda functions facilitated serverless, stateless operations—such as key share distribution and collection—ensuring high availability and minimal infrastructure management.
- The system achieved a balance between fault tolerance and responsiveness. Optimizations enabled key reconstruction and fund approval even in the presence of partial user participation, adhering to threshold cryptography principles.

9.5 Learnings and Reflections

Beyond technical accomplishments, the project offered valuable personal and academic growth:

- It enhanced our understanding of advanced cryptographic concepts such as Shamir's Secret Sharing, threshold recovery, and multi-party computation (MPC).
- We gained first-hand experience with blockchain ecosystems and smart contract development, enabling us to create secure digital interactions and decentralized workflows.
- Challenges like synchronizing cloud functions, ensuring communication security, and handling multi-user logic in smart contracts pushed us to explore creative problem-solving approaches.
- The collaborative nature of the project also taught us effective team coordination, documentation practices, and the importance of consistent testing and version control.

9.6 Final Remarks

This project stands as a testament to the potential of combining secure cryptographic protocols with decentralized blockchain-based control mechanisms. It demonstrates that a collaborative vault system can be built without centralized dependencies while maintaining strong fault tolerance and security guarantees. The project not only meets its intended goals but also opens pathways for innovations in secure multi-party applications, decentralized finance (DeFi), and digital trust systems. The experience has been enriching and foundational, equipping the team with both technical prowess and a problem-solving mindset that will be valuable in future research and professional endeavors.

CHAPTER 10

APPENDIX

10.1 Appendix A

10.1.1 Problem Statement Feasibility Assessment

10.1.1.1 Satisfiability Analysis

The core challenge of the multi-party crypto vault system is determining whether the system can securely process transactions, reconstruct cryptographic keys, and authorize actions based on multiple participants' input. This can be viewed as a satisfiability problem where the system must satisfy a set of constraints:

- All key holders must agree before the key can be reconstructed.
- Transactions must be authorized by a predetermined number of parties.

This problem can be modeled using Boolean satisfiability (SAT), where each decision (whether to reconstruct the key, authorize a transaction, etc.) is represented as a Boolean variable. The system is satisfiable if there exists a combination of decisions that meet the required conditions (e.g., quorum of key holders). By analyzing satisfiability, we ensure that the conditions for secure key reconstruction and transaction authorization are feasible under the given system constraints.

10.1.1.2 NP Classification and Computational Complexity

The multi-party vault system involves cryptographic key reconstruction, consensus-building among participants, and secure transaction authorization, which introduces significant computational complexity. To assess the problem's complexity:

- **NP-Hard Classification:** The problem of secure key reconstruction and transaction authorization in a multi-party setting can be classified as NP-Hard. This is due to the fact that validating whether a given transaction can be authorized (given the correct combination of key shares and signatures) requires checking a large number of possible combinations of key holders, which grows exponentially with the number of participants. The need to verify every possible combination makes the problem computationally expensive, similar to other NP-Hard problems like the Traveling Salesman Problem (TSP).
- **NP-Complete Classification:** In terms of decision problems, determining whether a transaction can be authorized (i.e., whether a valid

set of key shares exists that meets the threshold for key reconstruction) can be classified as NP-Complete. This is because the problem can be verified in polynomial time, but finding the correct combination of participants who satisfy the transaction authorization conditions requires solving a potentially exponential problem.

- **P-Type Classification (Certain Sub-Problems):** Some sub-problems in the system, such as verifying the validity of a cryptographic signature or validating individual key shares, are P-Type problems, meaning they can be solved in polynomial time. These computational tasks are efficiently solvable using modern cryptographic algorithms based on group theory, number theory, and algebraic structures.

10.1.1.3 Modern Algebra and Relevant Mathematical Models

The multi-party vault system leverages modern algebraic structures to enhance security and efficiency, primarily focusing on:

1. **Modular Arithmetic:** Used in cryptographic algorithms such as RSA and elliptic curve cryptography (ECC), which form the backbone of the key generation and signing processes
2. **Group Theory:** Applied in multi-signature schemes and threshold cryptography, where operations over algebraic groups (e.g., elliptic curve groups) ensure that only a specific number of participants can reconstruct a cryptographic key.
3. **Shamir's Secret Sharing:** The key reconstruction mechanism is based on Shamir's Secret Sharing Scheme, which is founded on polynomial interpolation over finite fields. This mathematical model allows the key to be split into shares that can only be reconstructed when a sufficient number of shares are combined.

10.2 Appendix B

Survey Paper Title

Multi-Party Crypto Vault System for Secure Digital Transactions

Conference Name

IEEE I2ITCON 2025

Authors

Rushi Balapure, Prachi Patil, Sameer Patil, Samiksha Sarode, Dr. S. N. Girme

Paper Abstract

This paper explores secure digital asset management via multi-party crypto vaults using Multi-Sig and MPC techniques. While Multi-Sig offers scalability, MPC provides stronger privacy by enabling encrypted authorization without key exposure. MPC is recommended for high-security applications, with future research focused on post-quantum cryptography and reducing MPC latency.

Reviewers' Comments

- **Reviewer 1:** "The paper addresses a timely topic in secure digital asset management, clearly comparing Multi-Sig and MPC techniques. The writing is clear and structured, and the discussion on fault tolerance and key management is useful. More detailed results would strengthen the contribution."
- **Reviewer 2:** "This work presents relevant insights into decentralized finance, especially the trade-offs between Multi-Sig and MPC. The topic is well-chosen, though the authors should clarify their contribution and improve presentation details to enhance impact."

Status

Accepted for Publication (To be presented at IEEE I2ITCON 2025)

Multi-Party Crypto Vault System for Secure Digital Transactions

Rushi Balapure
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
rushibalapure2003@gmail.com

Prachi Patil
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
prachimpail603@gmail.com

Sameer Patil
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
sameerspatil29@gmail.com

Samiksha Sarode
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
samiksha.sarode2003@gmail.com

Dr. Shital Girmé
Assistant Professor, Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
sngirme@pict.edu

Abstract—The significance of safe and cooperative digital asset management systems has been brought to light by the emergence of cryptocurrencies and decentralized finance (DeFi). By incorporating several parties in the administration and approval of transactions, multi-party crypto vaults offer a means to guarantee distributed control, privacy, and fault tolerance. The two main methods that support these vaults—Multi-Signature (Multi-Sig) and Multi-Party Computation (MPC), are reviewed in this study. While multi-sig schemes provide rapid scalable solutions for real-time applications, MPC provides robust privacy and security by permitting encrypted calculations without disclosing private keys. Furthermore, fault tolerance techniques like social recovery and time-locked protocols, as well as key management models like split-key and blockchain-based approaches, guarantee that vault systems continue to be safe and functional even in the event of participant failure. This paper recommends MPC as the optimal approach for high-security, privacy-sensitive applications, such as institutional custody and financial systems, while outlining avenues for future research, including post-quantum cryptography and latency reduction in MPC protocols.

Index Terms—Multi-Party Computation (MPC), Multi-Signature (Multi-Sig), Crypto Vaults, Threshold Cryptography, Blockchain Key Management, Decentralized Finance (DeFi), Threshold ECDSA, Social Recovery Mechanism, Time-Locked Withdrawal Protocol, Fault Tolerance in Cryptography, Split-Key Management, Smart Contracts, Privacy-Preserving Cryptography, Institutional Crypto Custody

I. INTRODUCTION

Cryptocurrency and blockchain technology have revolutionized digital asset management by allowing decentralized systems in which large amounts of assets are held by both people and businesses. The need for reliable and safe vault systems to stop unwanted access, reduce hazards, and guarantee long-term, sustainable management is growing along with the value and complexity of these assets. The dependency on a single keyholder in conventional custodial arrangements results in a single point of failure, making the system susceptible to loss, theft, or compromise. Multi-party crypto vaults divide power among several users in order to overcome these issues,

lowering dependency on any one person and boosting security via shared authority.

The primary challenge in creating a multipartycrypto vault is striking a balance between scalability, privacy, efficiency, and security. To address these demands, two well-known cryptographic techniques are frequently used: multi-signature (multi-sig) and multi-party computation (MPC). Multi-Sig is suitable for high-frequency operations in wallets and custodial systems since it necessitates many parties to sign and authorize transactions. However, because the the participants' keys are visible while signing, it restricts privacy. On the other hand, MPC protocols enable participants to jointly authorize transactions on encrypted data, preserving privacy and distributed trust, though with higher computational overhead and communication delays that complicate time-sensitive applications.

The operation of multi-party vaults depends on fault tolerance and key management in addition to cryptographic approaches. Although social recovery models and time-locked withdrawal protocols offer resilience in the event that some individuals lose keys or become unavailable, split-key management and blockchain-based key verification guarantee that no single member has complete control. Together, these systems guarantee that the vault will always be open and functional, even in the event of participant failure or misplaced keys.

The focus of current research on cryptography approaches is on improvements in key management, scalability, and privacy, all of which are essential for safe online transactions in multiparty crypto vaults. The methodology section presents fundamental cryptographic techniques, emphasizing Multi-Party Computation (MPC) and Multi-Signature (Multi-Sig) methods that allow for safe, multi-party authorization in vault systems. Building on these pillars, the suggested system combines blockchain-based key management, MPC, and a one-round threshold ECDSA to provide a strong solution that offers the high security, privacy, and fault tolerance required to protect digital assets.

Figure 10.1: Survey Paper

Implementation Paper Title

Multi-Party Crypto Vault System for Trustless Asset Protection

Conference Name

IEEE ICICNCT 2025

Authors

Rushi Balapure, Prachi Patil, Sameer Patil, Samiksha Sarode, Dr. S. N. Girme

Paper Abstract

Secure key management is a fundamental challenge in decentralized cryptographic systems. This project implements a multi-party crypto vault using threshold-based secret sharing to enhance security and eliminate single points of failure. The system distributes key shards across multiple parties, allowing reconstruction only through quorum-based collaboration. It integrates cryptographic protocols, secure computation, and blockchain smart contracts to enable trustless multi-party transactions. Security analysis covers collusion resistance, key compromise scenarios, and performance metrics. Experimental results confirm the system's efficiency, scalability, and suitability for decentralized finance and secure asset management.

Status

Submitted for Publication

Multi-Party Crypto Vault System for Trustless Asset Protection

Rushi Balapure
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
rushibalapure2003@gmail.com

Prachi Patil
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
prachimpail603@gmail.com

Sameer Patil
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
sameerspatil29@gmail.com

Samiksha Sarode
Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
samiksha.sarode2003@gmail.com

Dr. Shital Girmé
Assistant Professor, Department of Computer Engineering
Pune Institute of Computer Technology
Pune, India
sngirme@pict.edu

Abstract—Secure key management is a critical challenge in cryptographic systems, particularly in decentralized environments where trust assumptions must be minimized. This paper presents the implementation of a multi-party crypto vault that employs a threshold-based secret sharing mechanism to enhance security and resilience. Our approach distributes a cryptographic key into multiple shards, with each shard held by a separate party. The key is reconstructed only when a predefined quorum of parties collaborate, ensuring that no single entity has unilateral access to the complete key at any time. We provide a comprehensive analysis of the system architecture, including the mathematical formulation for shard generation and reconstruction. The implementation details highlight the use of cryptographic protocols and secure computation techniques to facilitate secure multi-party transactions. Additionally, we evaluate the system's security against potential attack vectors, including collusion, rogue participants, and key compromise scenarios. Experimental results demonstrate the efficiency of our approach in terms of computational overhead, transaction latency, and security guarantees. Our findings suggest that multi-party key management schemes offer a robust solution for secure transaction authorization in blockchain networks and other cryptographic applications.

Index Terms—Multi-Party Computation (MPC), Threshold Cryptography, Secret Sharing, Blockchain Security, Crypto Vaults, Decentralized Key Management, Smart Contracts, Shamir's Secret Sharing, Secure Multi-Party Transactions, Trustless Asset Protection, Cloud-based Key Reconstruction, Institutional Crypto Custody.

I. INTRODUCTION

Cryptographic key management is a fundamental challenge in modern security systems, particularly in decentralized and blockchain-based environments where trust minimization is essential. Traditional key management schemes rely on centralized entities or single custodians, creating single points of failure that are susceptible to theft, loss, or unauthorized access. To mitigate these risks, multi-party cryptographic vaults have emerged as a viable solution by distributing key ownership across multiple independent participants.

This paper presents the implementation of a multi-party crypto vault that employs a threshold-based secret sharing mechanism to enhance security and reliability. Our approach ensures that a cryptographic key is never fully exposed at any given time; instead, it is divided into multiple shards, with each shard held by a separate party. The key can only be reconstructed when a predefined number of authorized participants collaborate, preventing any single party from independently accessing or misusing the key.

The proposed system leverages secure multi-party computation (MPC) and cryptographic secret sharing techniques to facilitate secure transactions while maintaining decentralization. This approach is particularly relevant for blockchain-based applications, secure digital asset storage, and enterprise security frameworks requiring robust access control mechanisms. By integrating mathematical models for shard generation and reconstruction, we establish a cryptographic foundation that ensures security against various attack vectors, including collusion, rogue participants, and partial key exposure.

The remainder of this paper is structured as follows: Section 2 reviews related work on key management and secret sharing techniques. Section 3 presents the system architecture and implementation details. Section 4 introduces the mathematical formulation for shard generation and reconstruction. Section 5 provides a security analysis, while Section 6 details the experimental setup and performance evaluation. Finally, Section 7 discusses the results, and Section 8 concludes the paper with potential future improvements.

II. LITERATURE SURVEY

The literature survey discusses a number of proposals and existing, in-use cryptography and blockchain based solutions as they are focused on privacy, key management and scalability. Here, the use of sharding and cryptographic proofs to target scalability is highlighted, where nodes only maintain related transaction data while still validating transactions without

Figure 10.2: Implementation Paper

10.3 Appendix C

10.3.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith:

Similarity: 1%

Document Information	
Analyzed document	52 BE Project Report II.pdf (D210457996)
Submitted	2025-05-02 05:21:00 UTC+02:00
Submitted by	Shital
Submitter email	sssupase@gmail.com
Similarity	1%
Analysis address	sssupase.pict@analysis.arkund.com

Figure 10.3: Plagiarism Report

CHAPTER 11

REFERENCES

- [1] Leung, Derek Suhl, Adam Gilad, Yossi Zeldovich, Nickolai. (2019). Vault: Fast Bootstrapping for the Algorand Cryptocurrency.
- [2] Han, Jongbeen Song, Mansub Eom, Hyeonsang Son, Yongseok. (2021). An efficient multi-signature wallet in blockchain using bloom filter. 273-281. 10.1145/3412841.3441910.
- [3] A. A. T. Innocent and G. Prakash, "Blockchain Applications with Privacy using Efficient Multiparty Computation Protocols," 2019 PhD Colloquium on Ethically Driven Innovation and Technology for Society (PhD EDITS), Bangalore, India, 2019, pp. 1-3, doi: 10.1109/PhDED-ITS47523.2019.8986954. keywords: blockchain;privacy;multiparty computation;efficiency,
- [4] H. Gao, Z. Ma, S. Luo and Z. Wang, "BFR-MPC: A Blockchain-Based Fair and Robust Multi-Party Computation Scheme," in IEEE Access, vol. 7, pp. 110439-110450, 2019, doi: 10.1109/ACCESS.2019.2934147. keywords: Blockchain;Bitcoin;Smart contracts;Game theory;Robustness;Multi-party computation;blockchain;smart contract;fairness;robustness;secret sharing;game theory,
- [5] Z. Guan, X. Zhou, P. Liu, L. Wu and W. Yang, "A Blockchain-Based Dual-Side Privacy-Preserving Multiparty Computation Scheme for Edge-Enabled Smart Grid," in IEEE Internet of Things Journal, vol. 9, no. 16, pp. 14287-14299, 15 Aug.15, 2022, doi: 10.1109/JIOT.2021.3061107.keywords: Blockchain;Smart grids;Companies;Collaboration;Data privacy;Servers;Edge computing;Blockchain;data analytics;edge computing;privacy preservation;secure multiparty computation;smart grid,
- [6] A. Buchade and R. Ingle, "Performance analysis of key splitting algorithms for Cloud computing," 2016 International Conference on Computing, Analytics and Security Trends (CAST), Pune, India, 2016, pp. 562-566, doi: 10.1109/CAST.2016.7915031. keywords: Cryptography;Cloud computing;Computers;Electronic mail;NIST;Virtual machining;Threshold cryptography;Key management;Cloud computing;Key splitting;Key reconstruction
- [7] A. R. Buchade and R. Ingle, "Key Management for Cloud Data Storage: Methods and Comparisons," 2014 Fourth International Conference on Advanced Computing Communication Technologies, Rohtak, India, 2014, pp. 263-270, doi: 10.1109/ACCT.2014.78. keywords: Cloud

- computing;Servers;Organizations;Public key;Communities;Memory;Key management;cloud scenarios;onsite cloud;outsourced cloud;public cloud;symmetric key;applications,
- [8] Buchade, A. R. (2017). Algorithms and techniques for key management in cloud computing.
 - [9] Ji, Yunfeng Xiao, Yuting Gao, Birou Zhang, Rui. (2023). Threshold/Multi Adaptor Signature and Their Applications in Blockchains. *Electronics*. 13. 76. 10.3390/electronics13010076.
 - [10] Di Nicola V, Longo R, Mazzone F, Russo G. Resilient Custody of Crypto-Assets, and Threshold Multisignatures. *Mathematics*. 2020; 8(10):1773. <https://doi.org/10.3390/math8101773>
 - [11] Zhang, P., Ge, F., Liu, Y. (2023). Achieving maximum efficiency in schnorr-based multi-signature and applications in blockchain. *arXiv preprint arXiv:2305.13699*.
 - [12] Gennaro, R., Goldfeder, S. (2020). One round threshold ECDSA with identifiable abort. *Cryptology ePrint Archive*.
 - [13] Zhou J, Feng Y, Wang Z, Guo D. Using Secure Multi-Party Computation to Protect Privacy on a Permissioned Blockchain. *Sensors*. 2021; 21(4):1540. <https://doi.org/10.3390/s2104154>
 - [14] "Custody Protocols Using Bitcoin Vaults", *arXiv:2005.11776 [cs.CR]*
 - [15] Sahinbas, K., Catak, F. O. (2023). Secure multi-party computation-based privacy-preserving data analysis in healthcare IoT systems. In *Interpretable Cognitive Internet of Things for Healthcare* (pp. 57-72). Cham: Springer International Publishing.
 - [16] Chaeikar, S. S., Alizadeh, M., Tadayon, M. H., Jolfaei, A. (2022). An intelligent cryptographic key management model for secure communications in distributed industrial intelligent systems. *International Journal of Intelligent Systems*, 37(12), 10158-10171.
 - [17] Lu, K., Zhang, C. (2020, October). Blockchain-based multiparty computation system. In *2020 IEEE 11th International Conference on Software Engineering and Service Science (ICSESS)* (pp. 28-31). IEEE.
 - [18] Erinle, Y., Kethepalli, Y., Feng, Y., Xu, J. (2023). Sok: Design, vulnerabilities, and security measures of cryptocurrency wallets. *arXiv preprint arXiv:2307.12874*.

- [19] Boneh, D., Drijvers, M., Neven, G. (2018, October). Compact multi-signatures for smaller blockchains. In International Conference on the Theory and Application of Cryptology and Information Security (pp. 435-464). Cham: Springer International Publishing.
- [20] Khalil-Hani, M., Bakhteri, R. (2010, June). Securing cryptographic key with fuzzy vault based on a new chaff generation method. In 2010 International Conference on High Performance Computing Simulation (pp. 259-265). IEEE.